

動画で上肢の動きを測る

マーカースレスモーションキャプチャー入門 理学療法士のための半日ワークショップ

場面

外来で、肩が上がりにくい患者さんを担当している

決めたいこと

この4週間で、肩は前より上がるようになったか

ゴニオメーターは止めて測る

- 止まった姿勢の角度は、よく測れる
- 動いている途中の角度は、測れない

静的ROM評価 vs 動作中の実際の能力

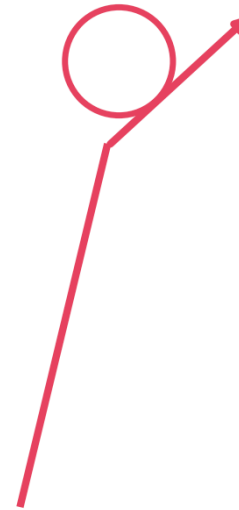
静的ROM評価：正常

肩屈曲150°
(ほぼ正常)



動作中（食事動作）：体幹代償・速度低下

代償的な
体幹前屈

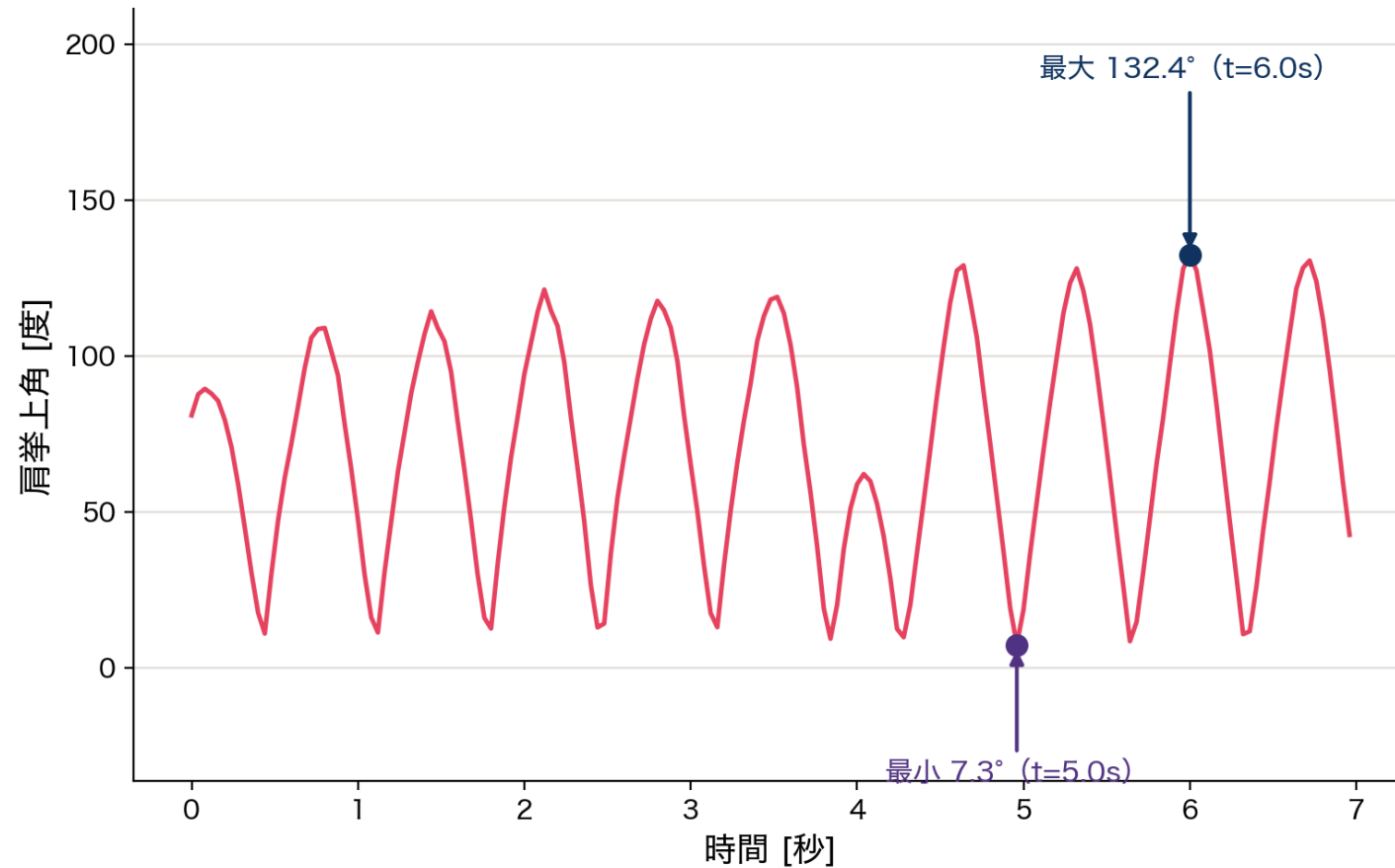


速度：健側の約30%
(動作中に低下)

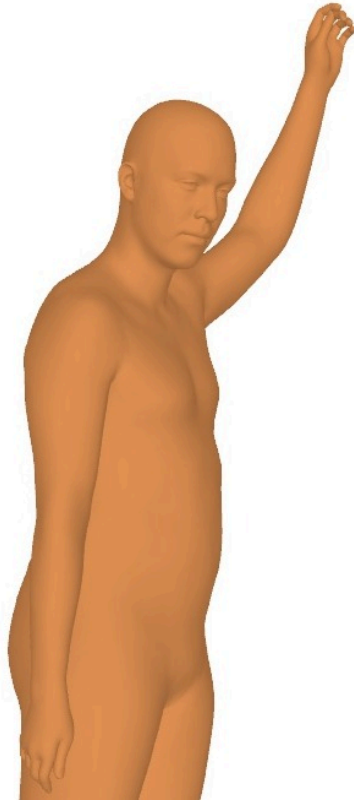
動画なら途中も残る

- 動画を撮れば、動きの全部のコマが残る
- 1コマごとに角度を出せば、動きが1本の線になる

右肩の挙上角の時系列 (サンプル実データ)



手元のデータ①：肩の挙上



公開されている運動動画（患者ではない）。真横から撮った前方挙上の繰り返し。7秒、1秒25コマ、計175コマ

手元のデータ②：注ぐ動作

- 公開されている動画
- 高齢の女性が、テーブルでグラスに水を注ぎ、手
に取る
- 8秒、1秒25コマ、計200コマ

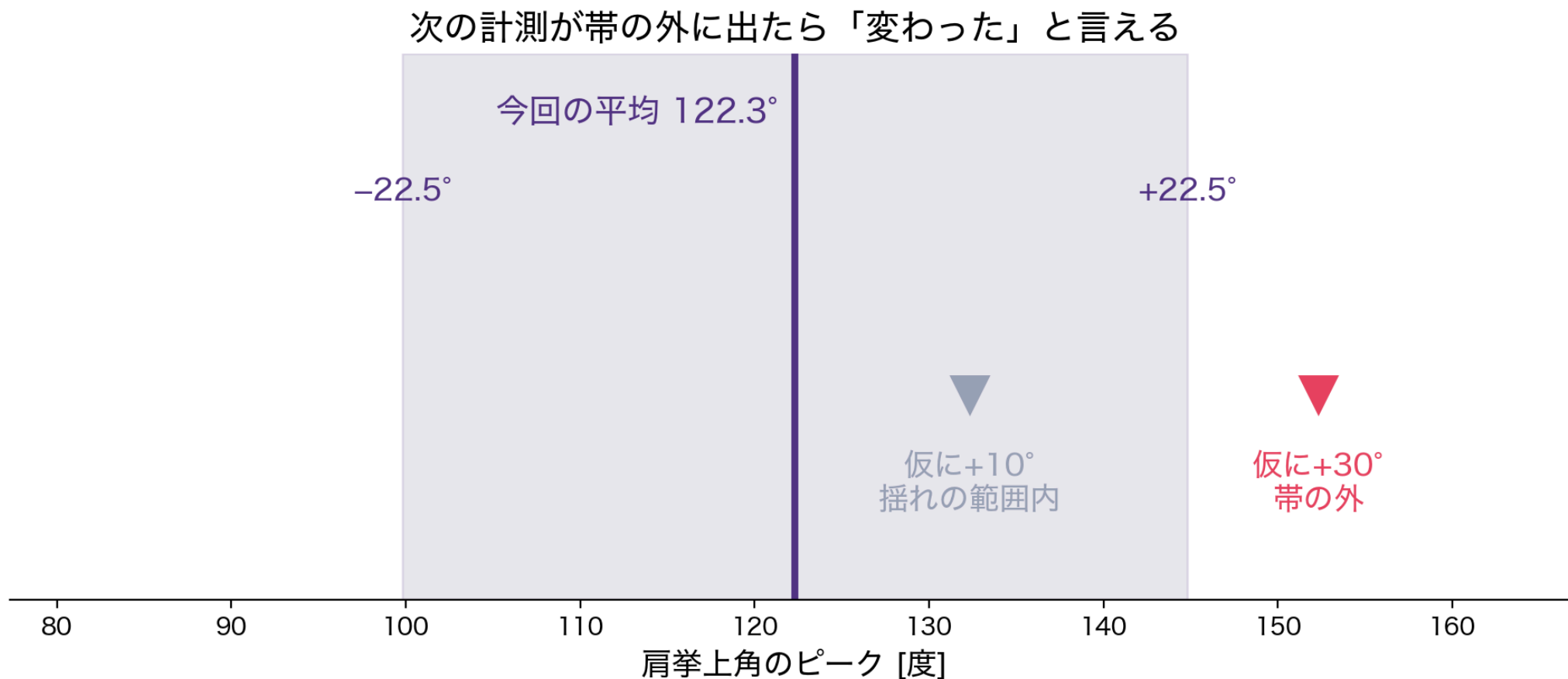


2本とも、すでに計算済み

動画から体の22点の位置を取り出した結果を、ファイルで配
っています

最後に出す答え

- 同じ撮り方で次に測ったとき、何度変われば「変わった」と言えるか



今日の問い

スマホ動画1本から、この上肢は良くなったと言えるか

今日の道筋

節	小さな問い
1	動画から何を取り出せるか
2	点から角度をどう出すか
3	実習①：自分で角度を出す
4	その数字は信じてよいか
5	どこまで、ではなく、どう動くか

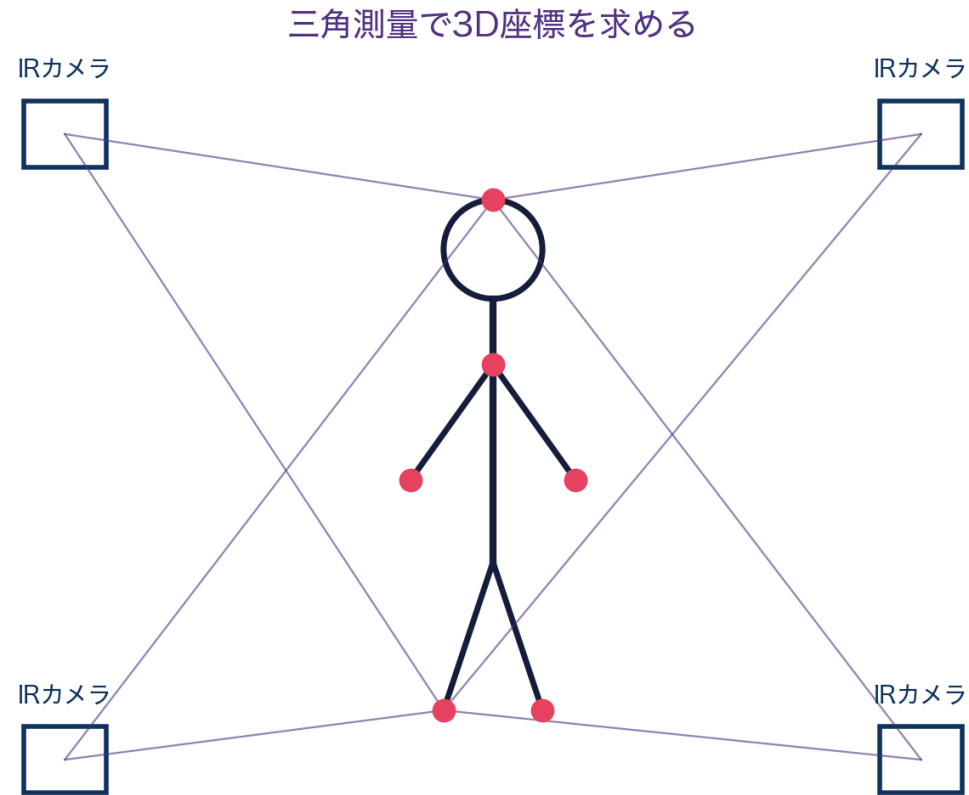
1. 動画から何を取り出せるか

角度を出すには、まず関節の位置がいる

マーカース式は体に印を貼る

- 反射する印を体に貼り、周りの赤外線カメラ数台で位置を測る

マーカース式MoCap：反射マーカース×赤外線カメラ

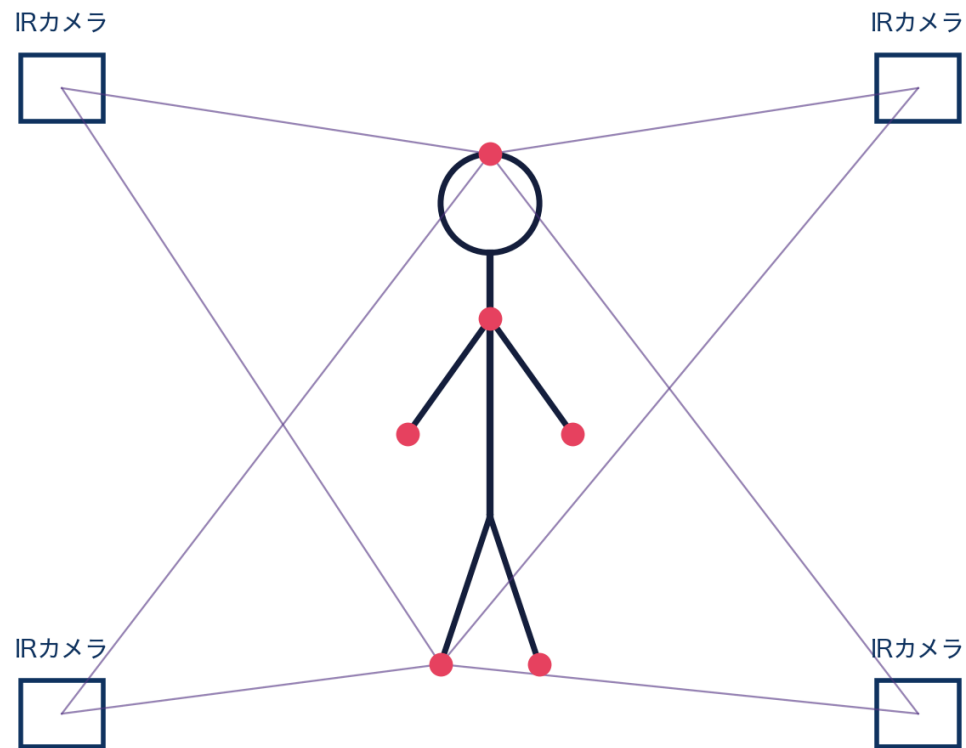


臨床ではそろえにくい

- 専用の部屋とカメラ、貼る手間、測る人が要る

マーカース式MoCap：反射マーカース×赤外線カメラ

三角測量で3D座標を求める



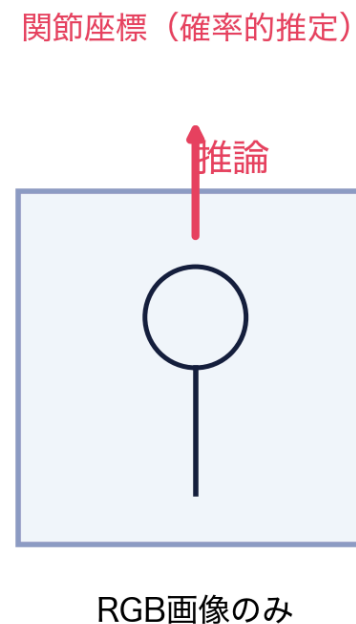
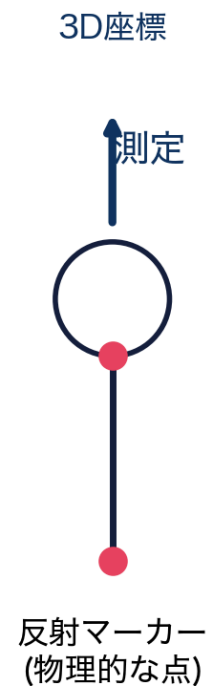
マーカースレスは画像から推定

- 印を貼らず、写っている画像から関節の位置を推定する

マーカース式 と マーカースレス の発想の違い

マーカース式 = 物理位置を測定

マーカースレス = 画像から推論



カメラ1台では奥行きが消える

- 正面から見ると、前に出した腕と横に出した腕が同じに見える

正面からの2D映像だけでは区別できない例

肩屈曲90°
(矢状面の動き)



肩外転90°
(前額面の動き)

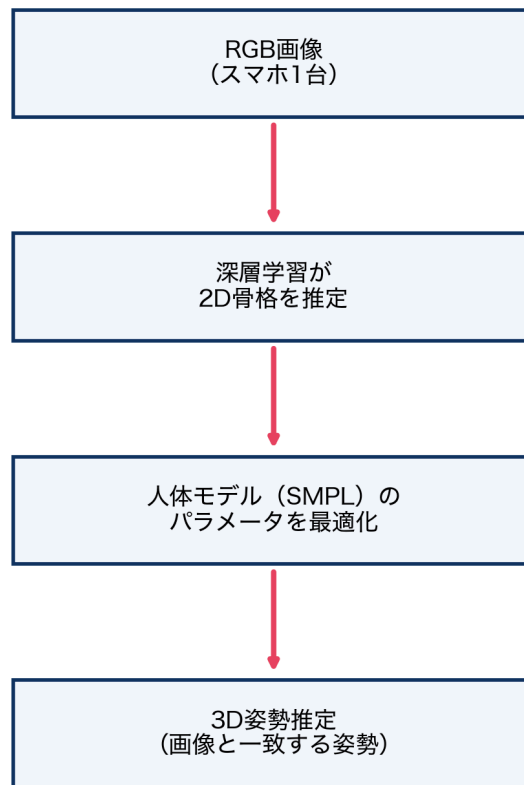


→ 正面カメラの2D画像では同じに見え、区別できない

奥行きは学習で補う

- 人の体の形をたくさん学んだモデルが、写っていない奥行きを推定する

単眼カメラによる3D姿勢推定の流れ



今日使うのはSAM-Body4D

- 動画の人物を追いかけて、1コマずつ体の形を推定する（2025年公開）

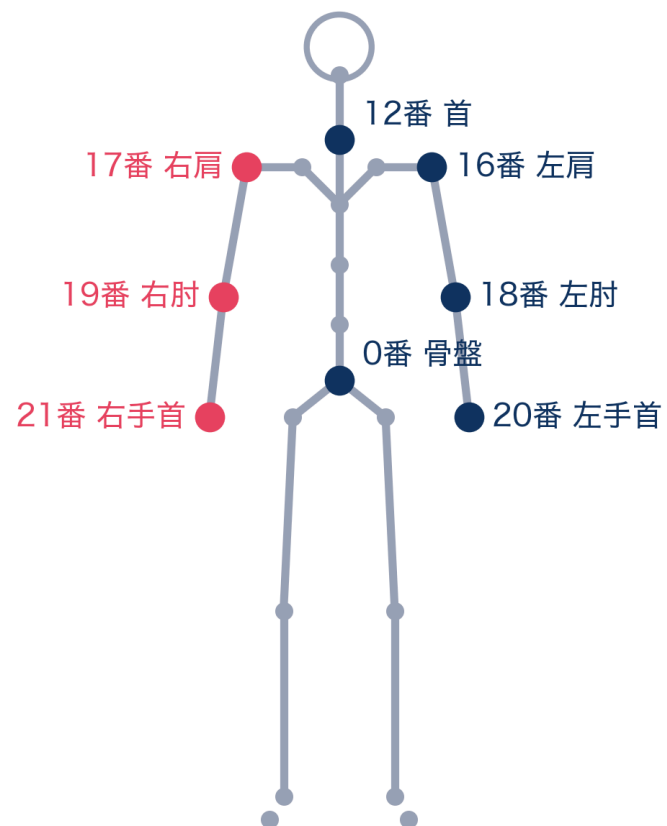
SAM-Body4D の処理の流れ（training-free）



入れるものと出てくるもの

- 入力：動画1本
- 出力：1コマごとに、体の22点の位置

体の22点（番号は0から21、模式図）



1つの点は3つの数

150コマ目の右肩と右肘（単位はメートル）

点	横	上下	前後
右肩（17番）	-0.083	1.509	0.062
右肘（19番）	-0.116	1.722	0.267

肘が肩より上・前にある。腕を前に上げたコマ

2. 点から角度をどう出すか

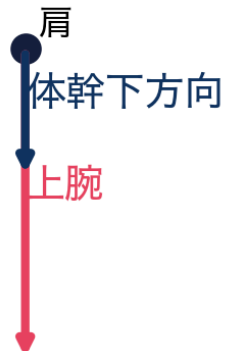
22点の位置は出た。では、点からどうやって角度を出すか

肩挙上角を振り返る

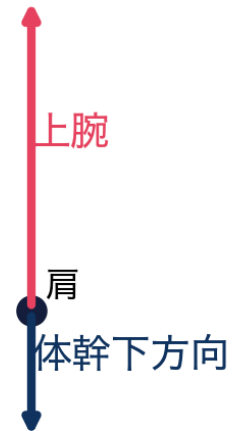
- 腕を下ろした位置が0°、真上が180°

肩挙上角の3態
挙上90°

挙上0°



挙上180°



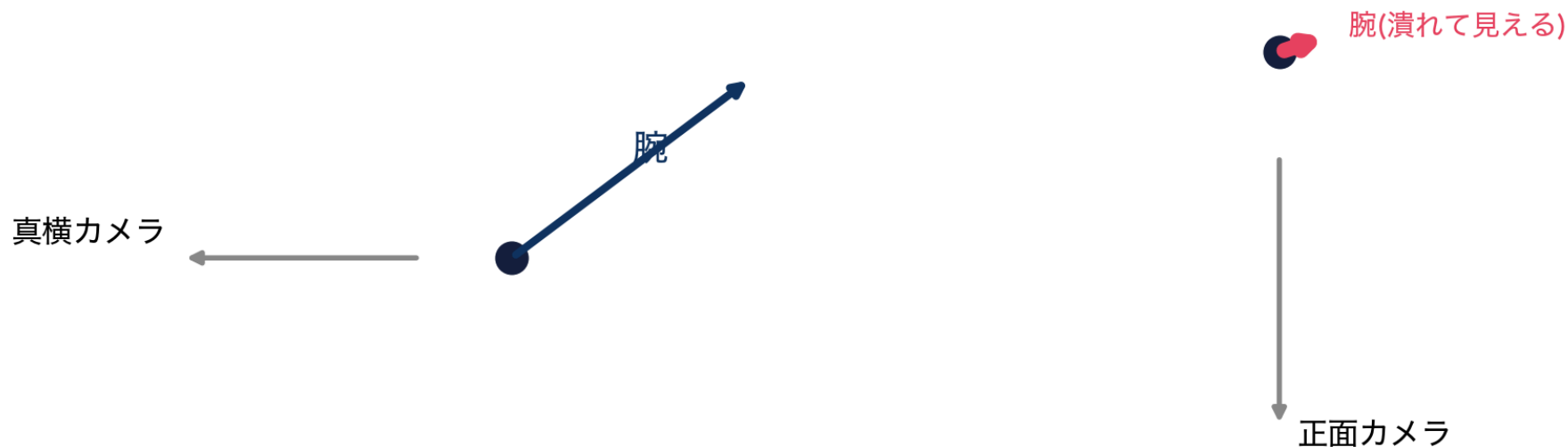
撮る向きで読み方が変わる

- 真横から撮れば屈曲、正面から撮れば外転として読む
- 今日の動画は真横から撮っているので、屈曲として読む

外転動作のカメラアングルによる見え方の違い

正面カメラ＝腕が潰れる (NG)

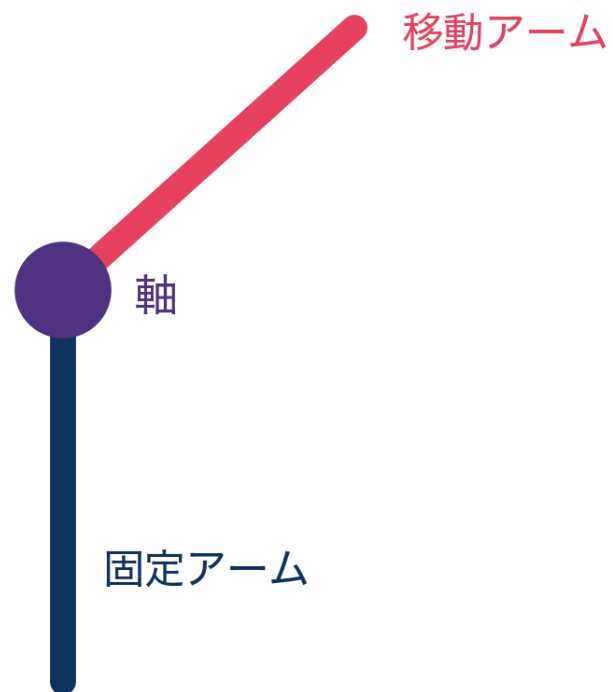
真横カメラ＝腕の開きが見える (OK)



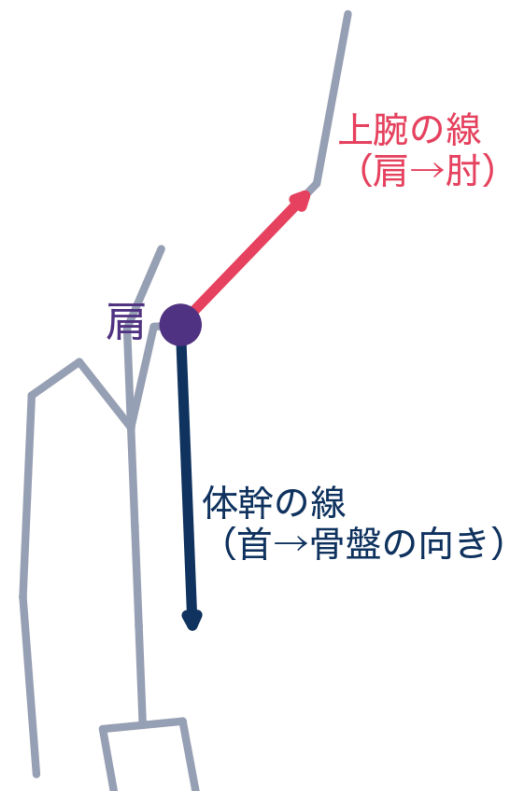
ゴニオメーターの3つの部品

- 軸を関節に、固定アームを体幹に、移動アームを上腕に当てる

ゴニオメーター



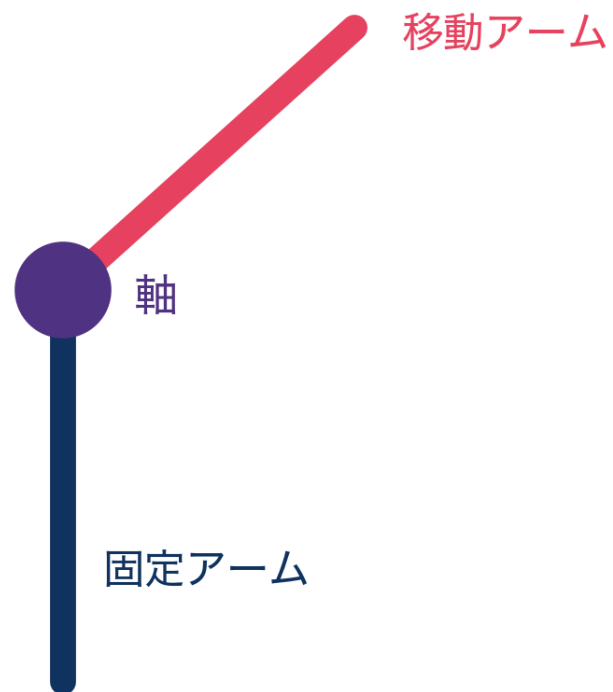
骨格（真横から見た1コマ）



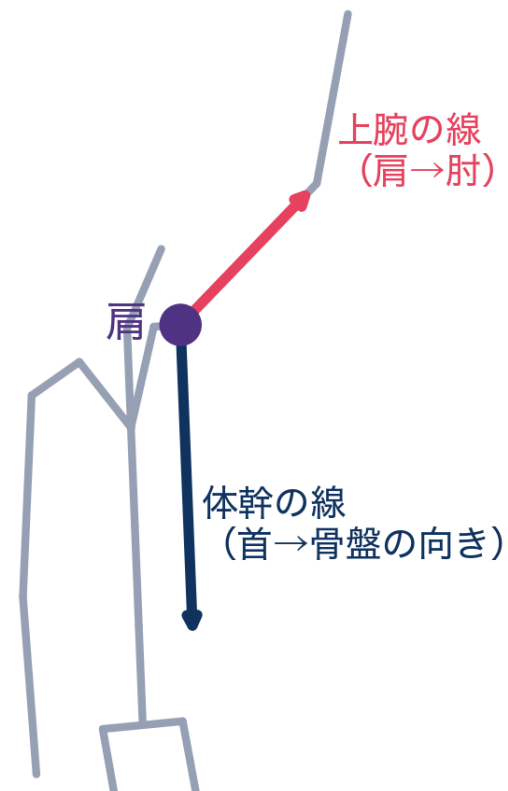
同じ当て方を骨格にする

- 軸は肩、固定アームは首→骨盤の線、移動アームは肩→肘の線

ゴニオメーター



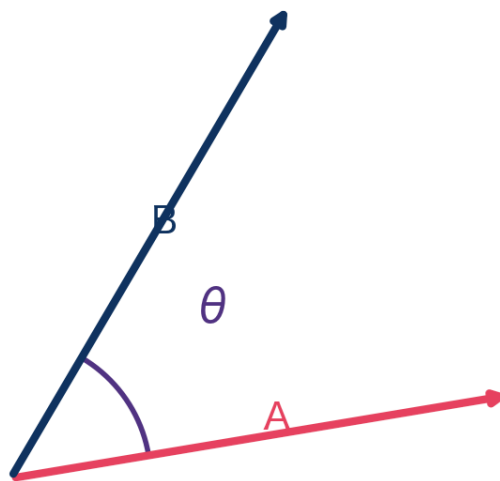
骨格（真横から見た1コマ）



2本の線は2本の矢印

- 点から点へ引いた矢印を「ベクトル」と呼ぶ
- 角度は、2本の矢印のなす角 θ

内積とベクトルのなす角 θ



$$\cos \theta = \mathbf{A} \cdot \mathbf{B} / (|\mathbf{A}| |\mathbf{B}|)$$

矢印2本から角度を出す式

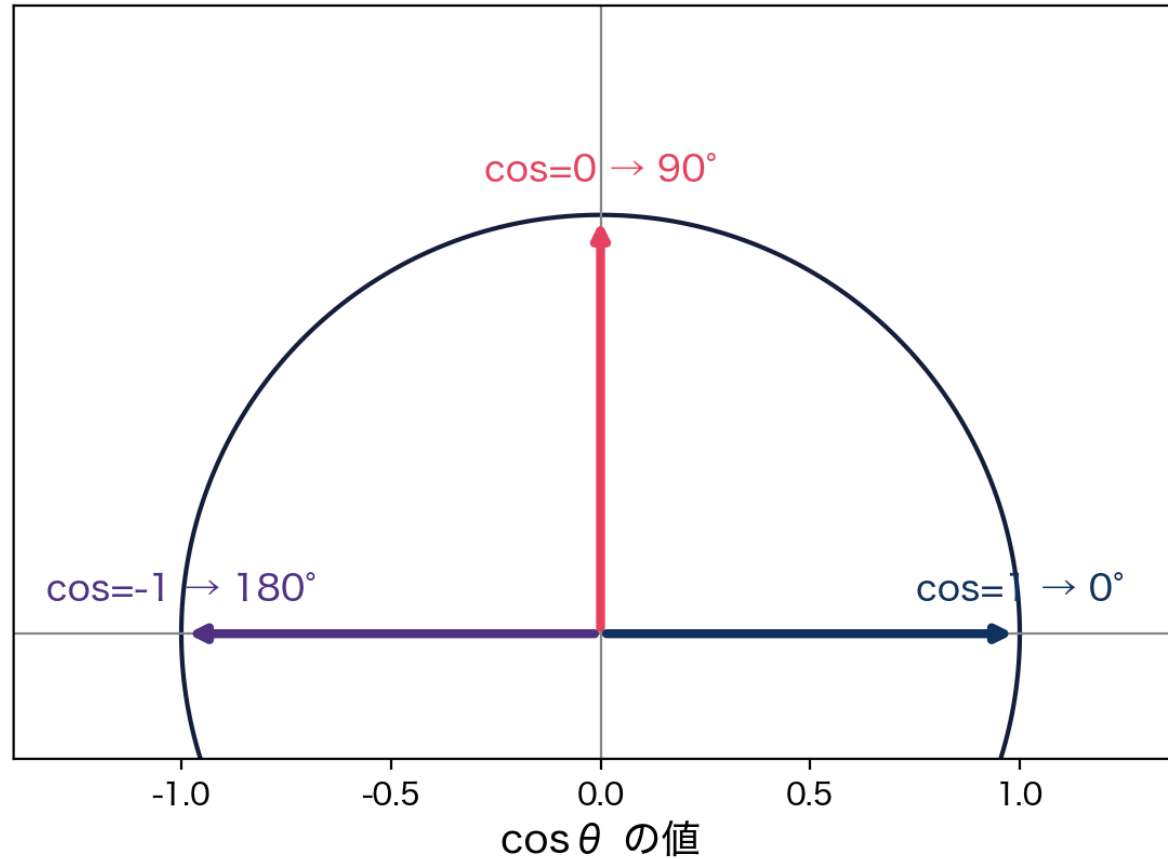
$$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|}$$

- $\vec{a}$: 上腕の矢印、 $\vec{b}$: 体幹の矢印
- 分子の $\vec{a} \cdot \vec{b}$ (内積) は、2本の向きのそろい具合
- 分母で、矢印の長さの影響を消す

cosの値から角度に戻す

- 向きがそろえば1で0°、直角なら0で90°、逆向きなら-1で180°

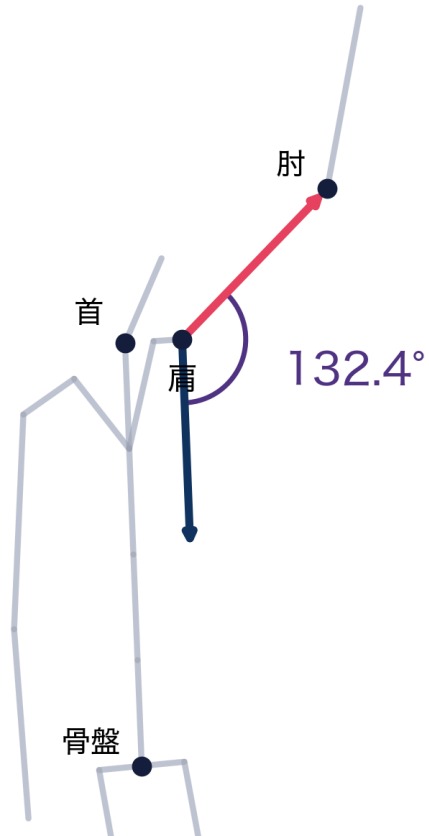
cosの値から角度を求める (arccos)



実データの1コマで測る

- 右肩がいちばん上がった150コマ目で、2本の矢印のなす角は 132.4°

実データ：右肩がいちばん上がったコマ（150コマ目）

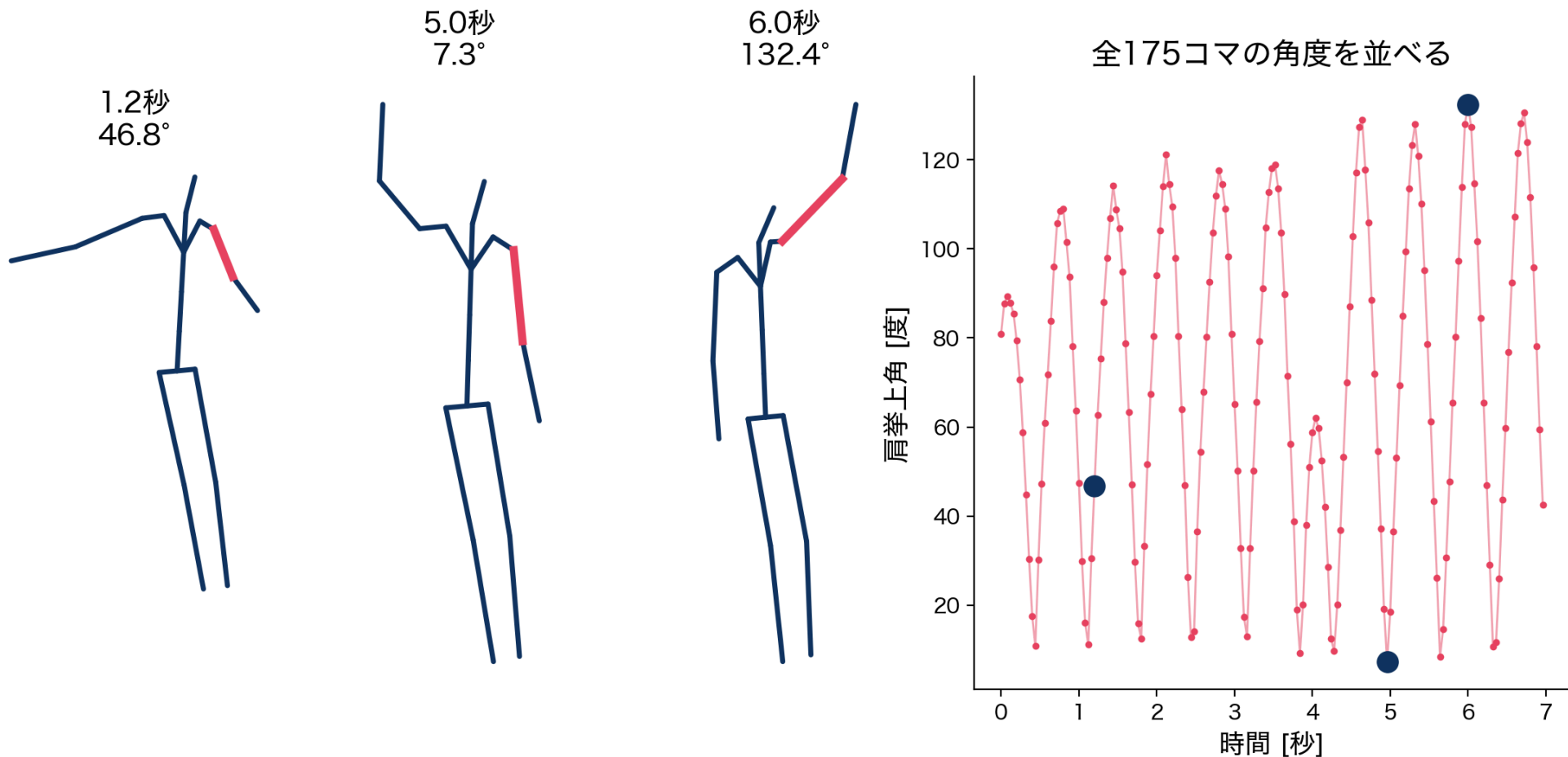


132.4°をどう読むか

真横から撮った動画なので、肩屈曲132.4°として読む

全部のコマで同じ計算をする

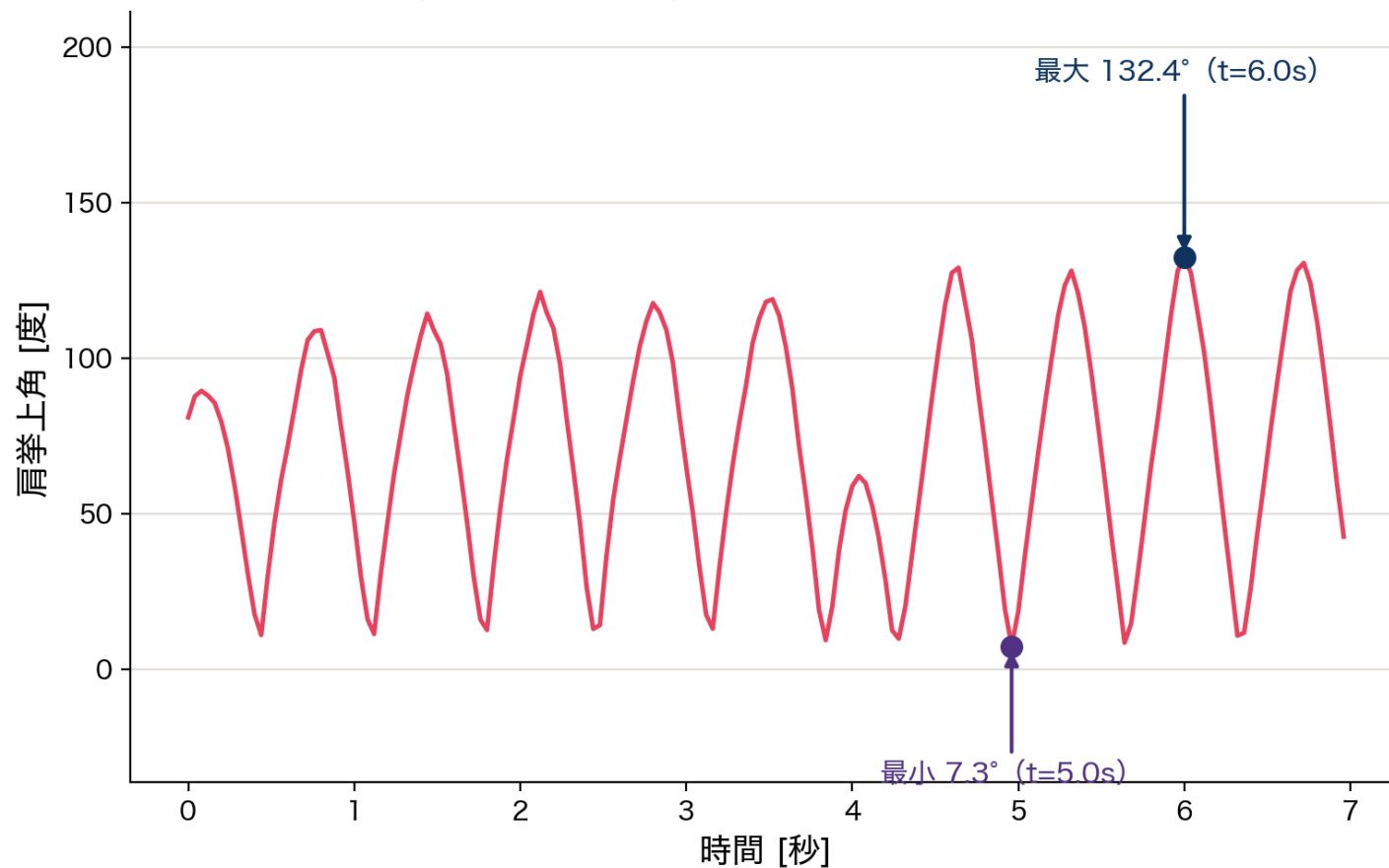
- 175コマすべてで同じ計算をして、時間の順に並べる



動きが1本の線になる

- 点を線で結ぶと、挙上の繰り返しが山と見える

右肩の挙上角の時系列 (サンプル実データ)



3. 実習①：自分で角度を出す

計算の流れは分かった。ここからは、同じ計算を自分のColabで動かす

コードはコンピューターへの手順書

- 評価の手順書と同じく、上から順に1行ずつ進む

評価の手順書（人が読む）

1. 端座位をとってもらう

2. 肩にゴニオメーターを当てる

3. 角度を読んで記録する

コード（コンピューターが読む）

```
data = json.load(f)
```

```
elev = compute_shoulder_elevation(...)
```

```
print(elev)
```

どちらも「上から順に1行ずつ」進む

いちばん短いコード

```
print('こんにちは')
```

- `print(...)` は、かっこの中を表示する命令
- `'` で囲んだものは、そのままの文字として扱われる

実行すると下に出る

```
print('こんにちは')
```

こんにちは

コードを動かすことを「実行する」と呼ぶ

セルは押すと動く箱

- Colabでは、コードを「セル」という箱に入れて、1つずつ実行する

セル：コードを入れる箱



```
print(1 + 2)
```

3

ここを押す

← ▶を押すと、すぐ下に結果が出る

ノートを開く

1. 配られたリンクを開く（Googleアカウントでログイン）
2. 左上の［ドライブにコピー］を押す
3. 「0. 最初の1行」のセルの  を押し、「こんにちは」が出るのを確かめる

約束

上のセルから順に、全部押す。下のセルは、上のセルで作ったものを使う

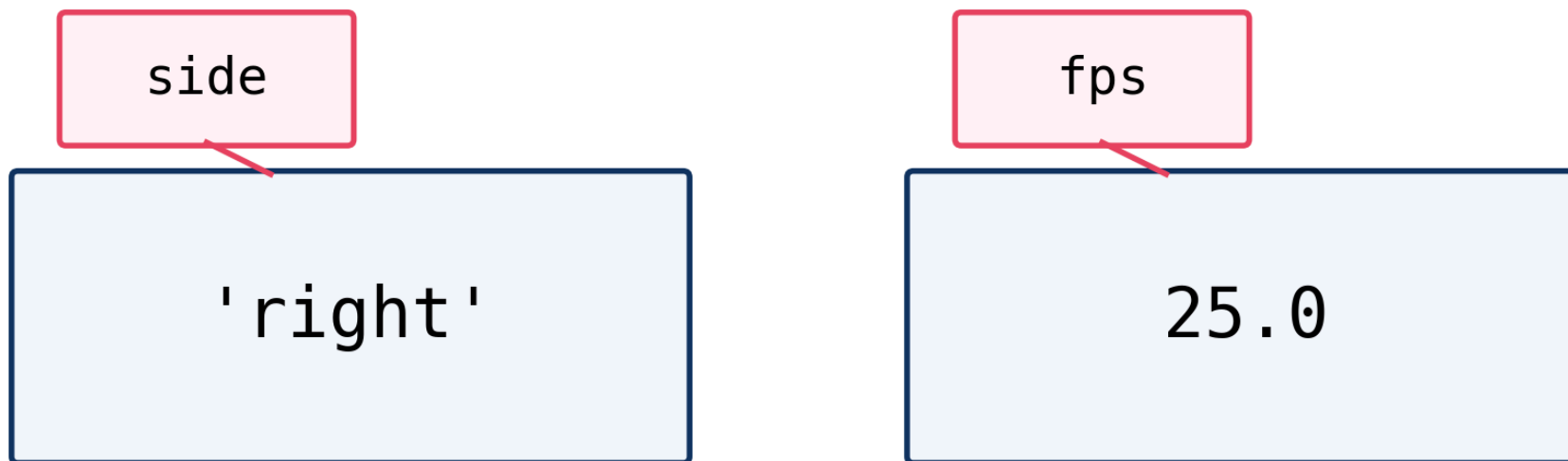
道具箱を開ける

```
!pip install -q matplotlib-fontja
import json
import numpy as np
import matplotlib.pyplot as plt
import matplotlib_fontja
from scipy.signal import find_peaks
print('道具の準備ができました')
```

- `import` は、計算やグラフの道具が入った箱を開ける命令
- 1行目は、グラフに日本語を出す道具を入れる行（`!` はColabへの頼みごと）

変数は値に付けた名札

- 値に名前を付けて取っておくと、あとで名前を書くだけで取り出せる



名前（名札）を書けば、中の値を何度でも取り出せる

最初の変数：配られたID

```
FILE_ID = 'YOUR_FILE_ID_HERE'  
!gdown {FILE_ID} -O rom_task_output.json
```

- 1行目：配られたIDに、`FILE_ID` という名札を付ける
- `=` は「等しい」ではなく「右の値に左の名前を付ける」
- 2行目：そのIDのファイルを持ってくる

やってみよう

' と ' の間の文字を、配られたIDに書き換えて ▶ を押す。引用符 ' は消さない

ファイルを開いてしまう

```
with open('rom_task_output.json') as f:  
    data = json.load(f)  
print('データを開きました')
```

- 上の2行はひとかたまりで読む：ファイルを開き、中身に `data` という名札を付ける
- 2行目の字下げ（行頭の空白）は、上の行のかたまりに入っている印

中身の大きさを見る

```
joints = np.array(data['joints'])  
fps = data['meta']['fps']  
n_frames = len(joints)  
print('コマ数', n_frames)  
print('1秒あたりのコマ数', fps)  
print('動画の長さ（秒）', n_frames / fps)
```

```
コマ数 175  
1秒あたりのコマ数 25.0  
動画の長さ（秒） 7.0
```

名前で取り出す

- `data['joints']` : `data` の中の、`joints` という名前の中身 (22点の位置)
- `data['meta']['fps']` : `meta` の中の `fps` (1秒あたりのコマ数)
- `len(...)` : 中にいくつ入っているか (ここではコマの数)

番号で取り出す

```
print('右肩', np.round(joints[150][17], 3))  
print('右肘', np.round(joints[150][19], 3))
```

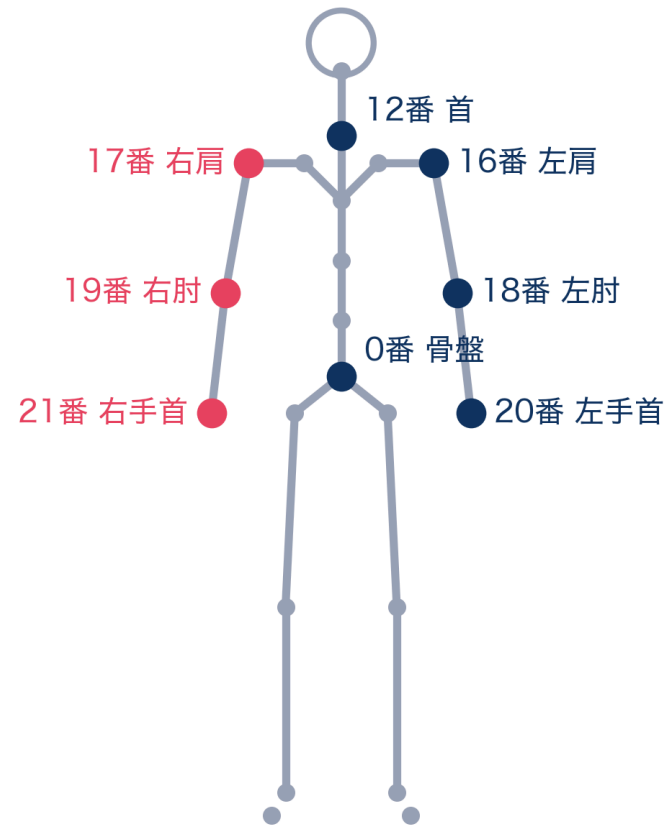
```
右肩 [-0.083  1.509  0.062]  
右肘 [-0.116  1.722  0.267]
```

`joints[150][17]` は、150コマ目の17番の点。番号は0から数える

17番は右肩、19番は右肘

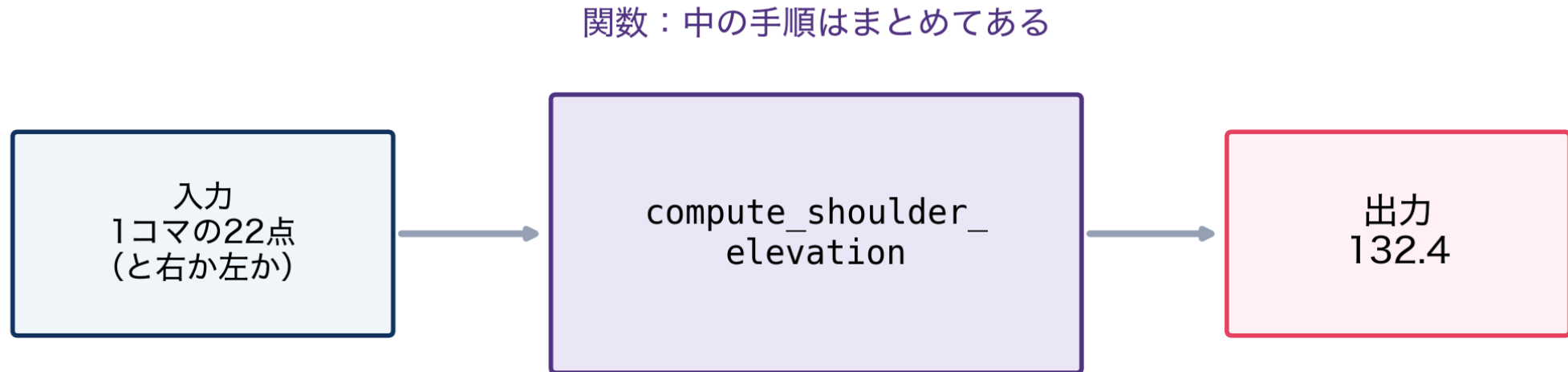
- 1節で見た22点の番号を、そのまま使う

体の22点（番号は0から21、模式図）



関数は入力を結果に変える機械

- 中の手順をまとめておき、入力を渡すと結果が返ってくる



使う人は「何を入れると、何が出るか」だけ知っていればよい

関数の形

```
def 関数の名前(入力):  
    手順  
    return 結果
```

- `def` で始め、`return` で結果を返す
- 手順は字下げして、かたまりにする

この枚は形の説明で、ノートにはない

2本の矢印のなす角の関数

```
def angle_between_vectors(v1, v2):  
    v1 = v1 / np.linalg.norm(v1)  
    v2 = v2 / np.linalg.norm(v2)  
    cos_angle = np.clip(np.dot(v1, v2), -1.0, 1.0)  
    return np.degrees(np.arccos(cos_angle))
```

- 役割：2本の矢印のなす角を出す（2節の式そのもの）
- 入力：2本の矢印 `v1` と `v2` 出力：角度（度）
- 中身は読み飛ばしてよい

押しても何も出ない

関数は、作っただけでは動かない。呼んだときに動く

肩挙上角の関数

```
def compute_shoulder_elevation(frame, side='right'):
    # 1. どの点を使うか（番号は0から数える）
    if side == 'right':
        shoulder, elbow = frame[17], frame[19]
    else:
        shoulder, elbow = frame[16], frame[18]
    neck, pelvis = frame[12], frame[0]
    # 2. 2本の線を作る
    upper_arm = elbow - shoulder    # 移動アーム
    trunk = pelvis - neck           # 固定アーム
    # 3. なす角を返す
    return angle_between_vectors(upper_arm, trunk)
```

3つのかたまりで読む

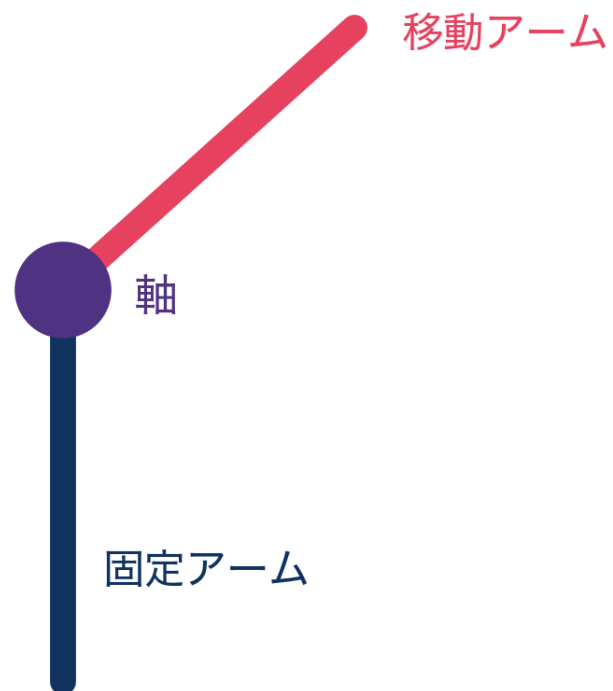
1. どの点を使うか：右なら17番と19番、左なら16番と18番
2. 2本の線を作る：点から点を引くと、矢印になる
3. なす角を返す：さっきの関数に2本の矢印を渡す

`#`から行末までは、人のためのメモ（コメント）。実行されない

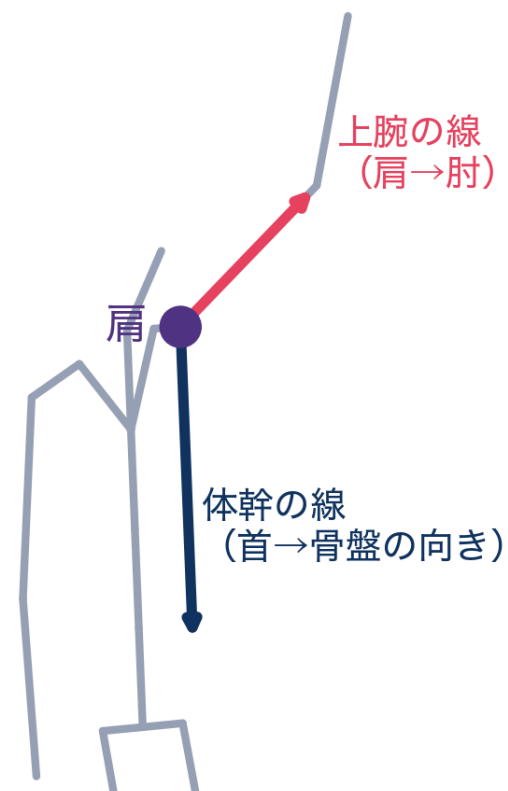
ゴニオメーターと同じ当て方

- 軸は肩、固定アームは首→骨盤、移動アームは肩→肘

ゴニオメーター



骨格（真横から見た1コマ）



関数を呼ぶ

```
angle = compute_shoulder_elevation(joints[150], 'right')  
print('150コマ目の肩挙上角', round(angle, 1))
```

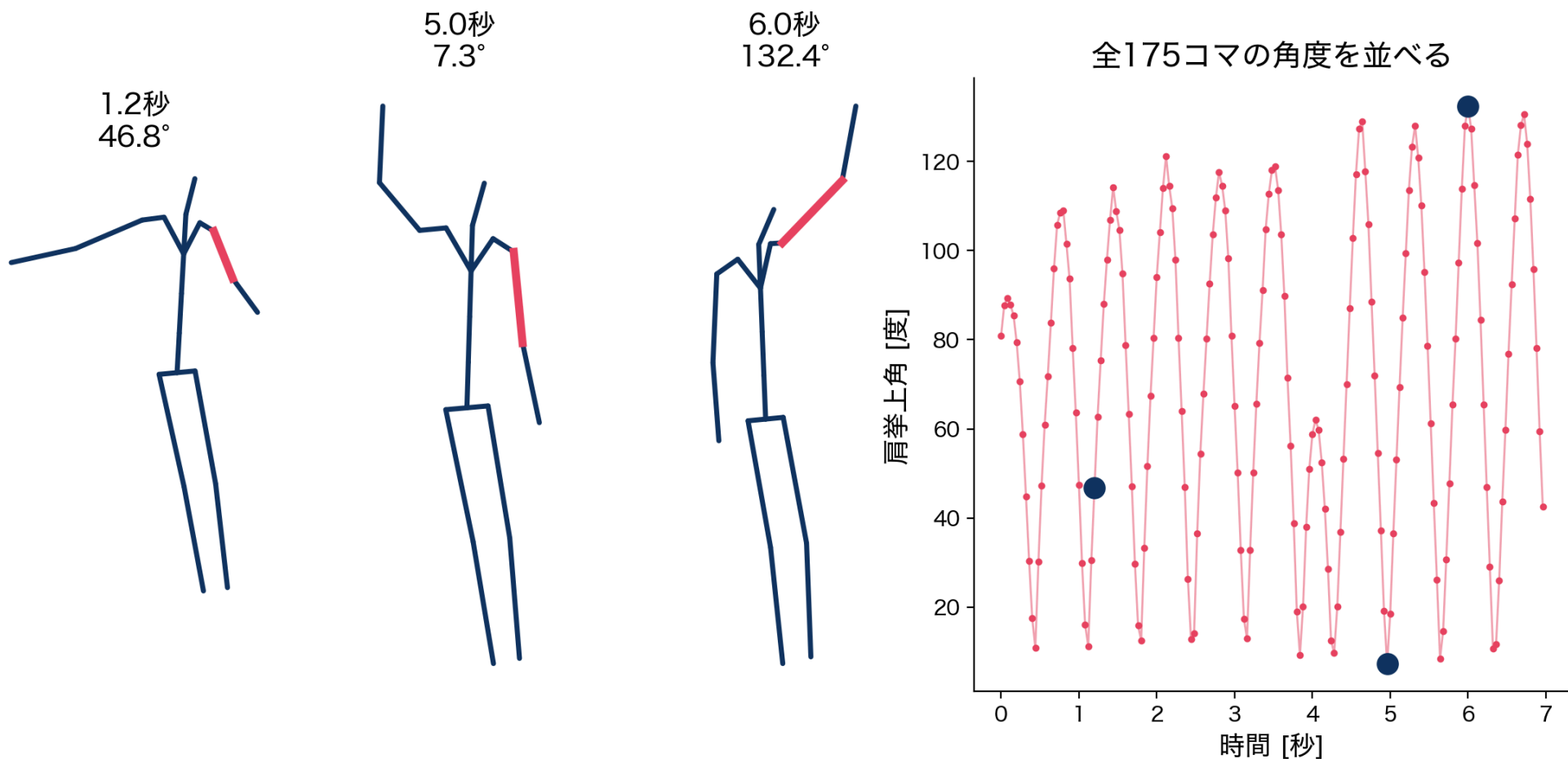
150コマ目の肩挙上角 132.4

- 関数の名前(入力) と書くと、関数が動いて結果が返る
- 返ってきた角度に、 angle という名札を付けた

2節で図を描いて出した 132.4° と、同じ値が返ってきた

175コマぶん繰り返したい

- 1コマで測れた。同じ測定を、全部のコマで繰り返す



繰り返しの命令for

```
side = 'right'
shoulder_elev = []
for i in range(n_frames):
    a = compute_shoulder_elevation(joints[i], side)
    shoulder_elev.append(a)
shoulder_elev = np.array(shoulder_elev)
print('最大', round(shoulder_elev.max(), 1))
print('最小', round(shoulder_elev.min(), 1))
```

最大 132.4
最小 7.3

1行ずつ読む

- `shoulder_elev = []` : 空の入れ物を用意する
- `for i in range(n_frames):` : `i` を0、1、2...174と変えながら、字下げした行を繰り返す
- `.append(a)` : 測った角度を入れ物の最後に足す

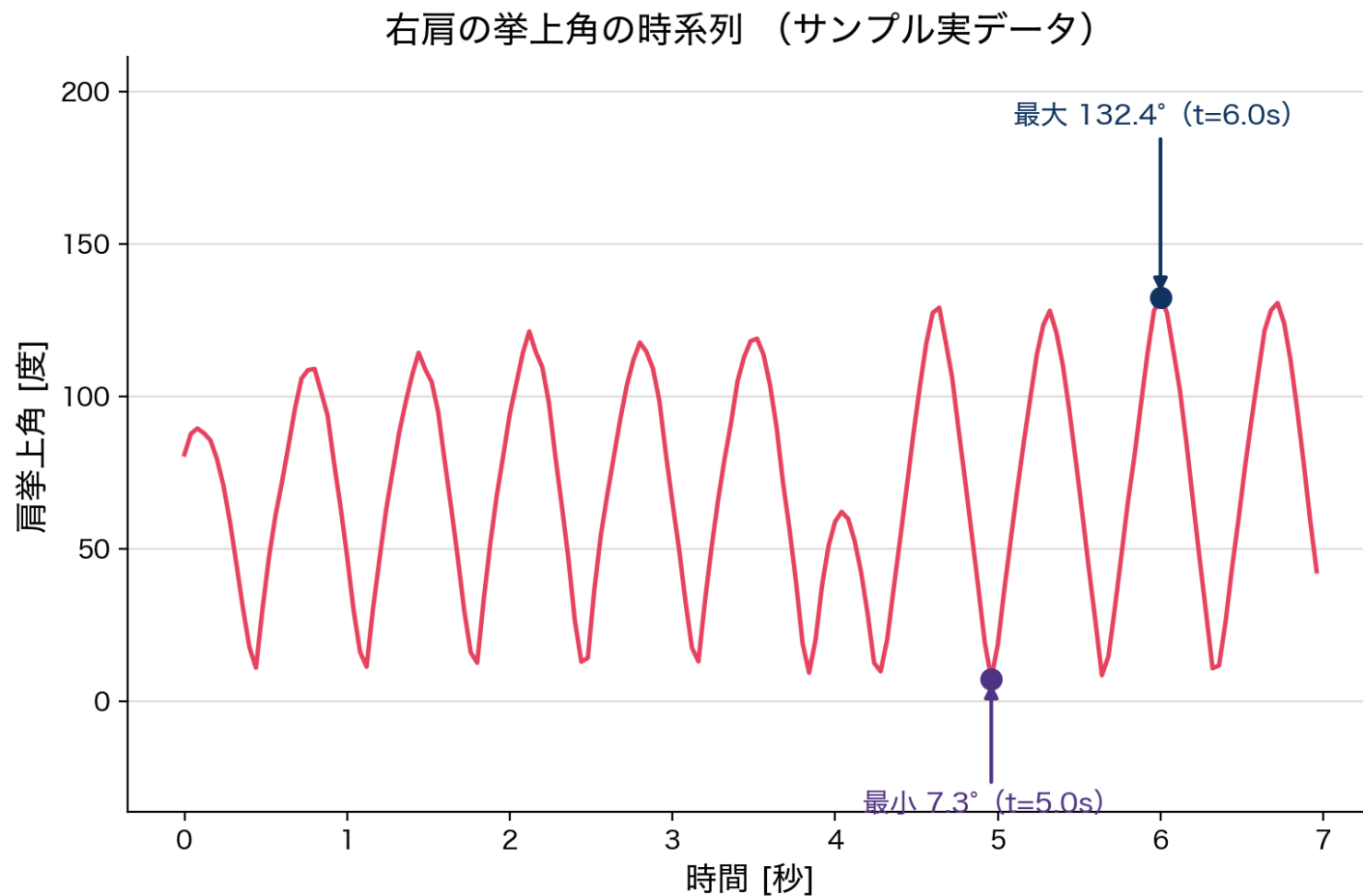
グラフを描く命令

```
time = np.arange(n_frames) / fps
plt.plot(time, shoulder_elev)
plt.xlabel('時間 [秒]')
plt.ylabel('肩挙上角 [度]')
plt.show()
```

- 1行目：コマの番号を1秒あたりのコマ数で割り、時間（秒）にする
- `plt.plot(横, 縦)` で線を引き、軸に名前を付け、`plt.show()` で表示する

出てくるグラフ

- 2節で見た動きの線と同じものが、自分のColabに出る



肘も同じ形で測る

```
def compute_elbow_flexion(frame, side='right'):
    if side == 'right':
        shoulder, elbow, wrist = frame[17], frame[19], frame[21]
    else:
        shoulder, elbow, wrist = frame[16], frame[18], frame[20]
    inner = angle_between_vectors(shoulder - elbow, wrist - elbow)
    return 180.0 - inner

elbow_flex = []
for i in range(n_frames):
    elbow_flex.append(compute_elbow_flexion(joints[i], side))
elbow_flex = np.array(elbow_flex)
print('肘屈曲の最大', round(elbow_flex.max(), 1))
```

肘屈曲の最大 99.9

肘の関数を読む

- 軸は肘、2本の矢印は肘→肩と肘→手首
- 2本のなす角は、伸ばすと 180° 。 180° から引くと、臨床の屈曲角（伸ばすと 0° ）
- 下半分は、肩と同じ「関数を作る → 全コマで繰り返す」の形

やってみよう：左肩を測る

1. 「9. 全コマを測る」のセルで、`side = 'right'` を `side = 'left'` に書き換える
2. そのセルから下を、全部押し直す
3. 最大の角度と、時系列のグラフの形を、右と比べる

答え合わせ

	最大	最小
右肩	132.4°	7.3°
左肩	142.6°	5.4°

同じ動画、同じ計算でも、右と左で最大は10°ほど違う

困ったとき

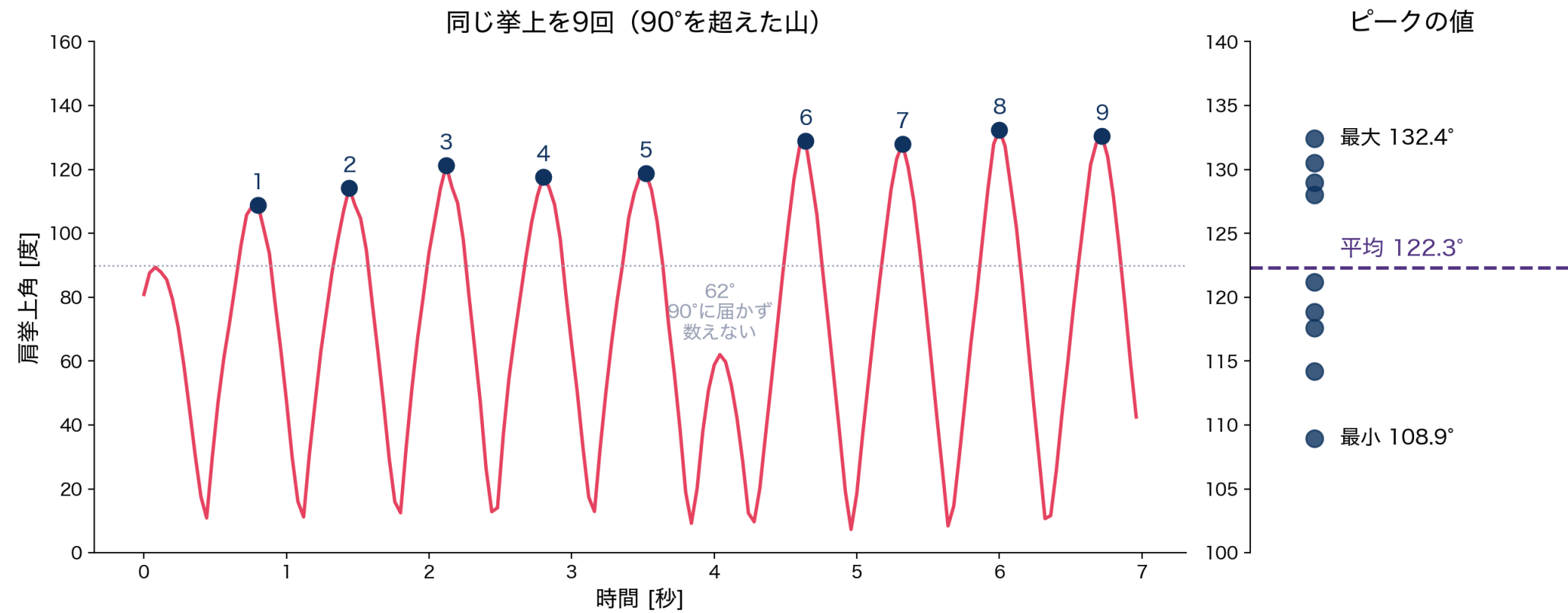
出たもの	よくある理由	直し方
<code>NameError</code>	上のセルを押していない	[ランタイム] → [すべてのセルを実行]
<code>SyntaxError</code>	引用符 <code>'</code> やかっこを消した	書き換えた所を元の形に戻す
グラフの文字が □	1のセルを押していない	1のセルから押し直す

4. その数字は信じてよいか

角度は出せた。では、その数字は信じてよいか

同じ拳上なのに、ピークは毎回違う？

- 7秒の動画の中で、同じ拳上を何度も繰り返している



ピークを数えるコード

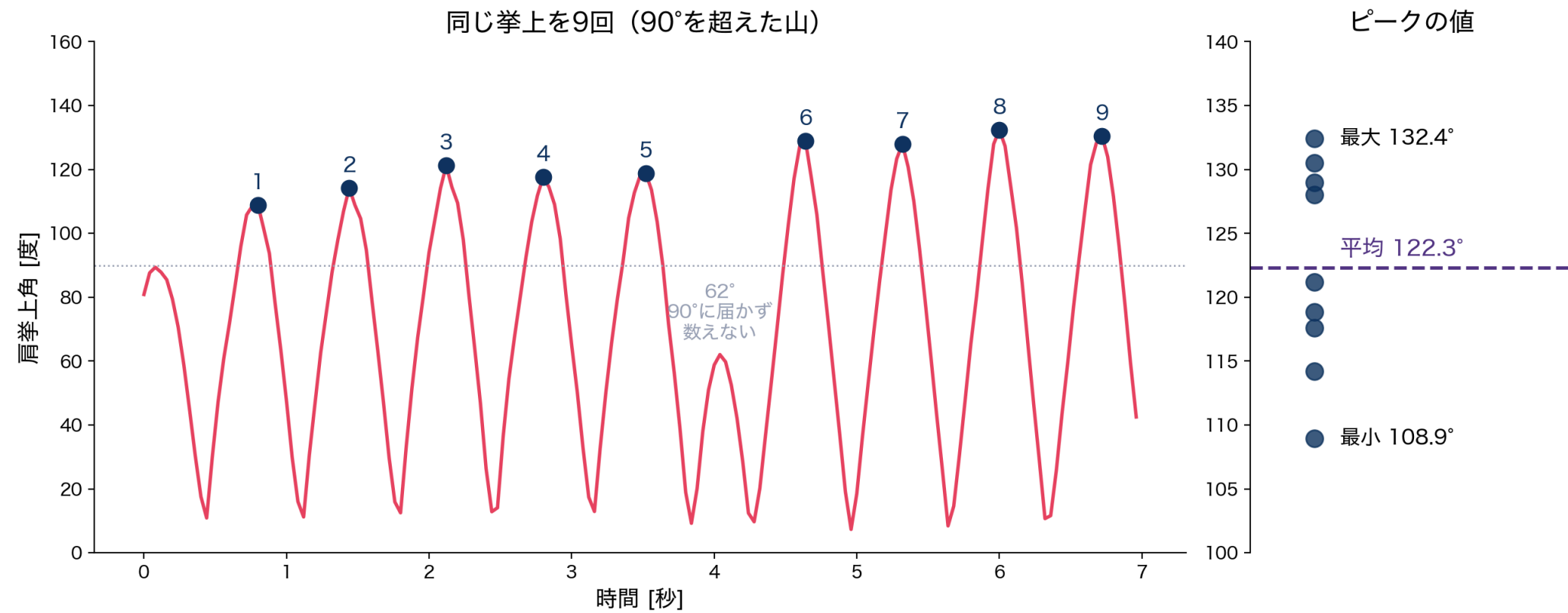
```
peaks, _ = find_peaks(shoulder_elev, height=90, prominence=30)
peak_values = shoulder_elev[peaks]
print('挙上の回数', len(peaks))
print('毎回のピーク', np.round(peak_values, 1))
```

```
挙上の回数 9
毎回のピーク [108.9 114.2 121.2 117.6 118.9 129. 128. 132.4 130.5]
```

- `find_peaks` は山を探す道具。90°を超えた山だけを数える
- 答えを2つ返すが、使うのは1つ目だけなので、2つ目は `_` で受けて捨てる

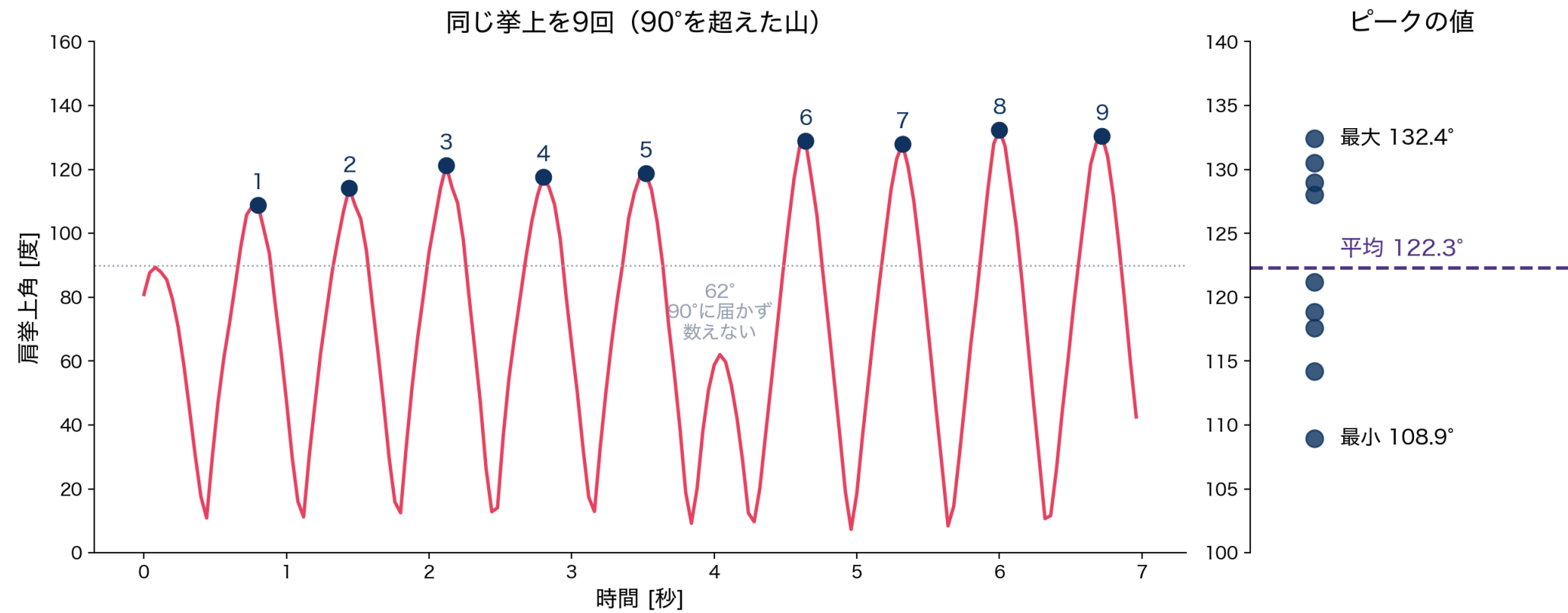
9回のピークは108.9°~132.4°

- いちばん低い回と高い回で、23.5°違う



途中で止めた1回は数えない

- 4秒あたりの62°の山は、90°に届いていないので、挙上の回に数えない



揺れには2つの理由が混ざる

揺れの理由	例
本人の動きが毎回違う	前半は低く、後半は高く上げている
推定が毎回少しずれる	同じ姿勢でも、コマごとに点の位置が揺れる

この動画だけでは、2つを分けられない

測定の揺れを表す数：SEM

- 同じものを繰り返し測ったときの、値の揺れの大きさ
- 測定誤差（standard error of measurement）と呼ぶ
- 同じ人の繰り返しの標準偏差（SD）が、その目安になる

変わったと言える最小の幅：MDC

$$\text{MDC}_{95} = 1.96 \times \sqrt{2} \times \text{SEM}$$

- 最小可検変化量（minimal detectable change）と呼ぶ
- 変化がこの幅を超えたら「測定の揺れではない」と、95%の確からしさで言える

式を読む

- $\sqrt{2}$: 比べるには2回測る。2回とも揺れるので、揺れが2回ぶん入る
- 1.96 : 揺れの95%が入る幅にするための数
- SEMが小さい測り方ほど、MDCも小さい

式をコードに写す

```
mean = peak_values.mean()
sd = peak_values.std(ddof=1)
mdc95 = 1.96 * np.sqrt(2) * sd
print('平均', round(mean, 1))
print('SD', round(sd, 1))
print('MDC95', round(mdc95, 1))
```

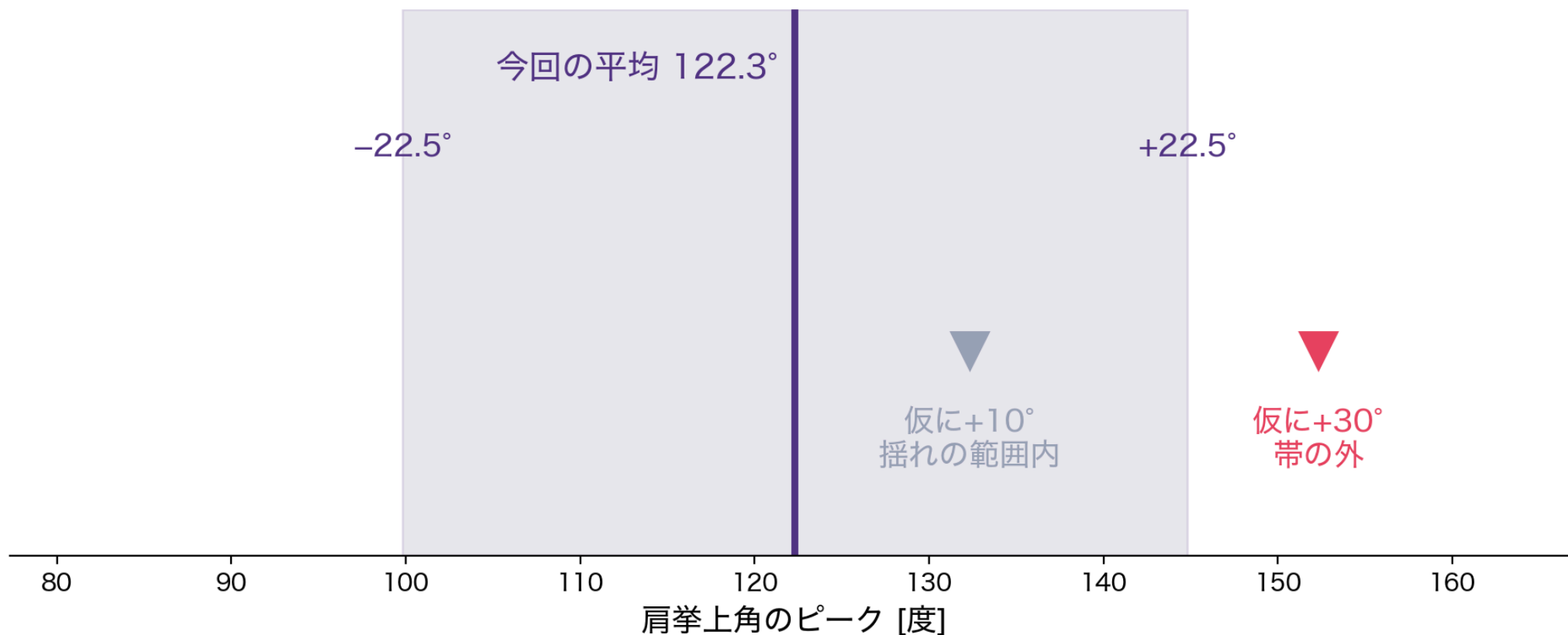
```
平均 122.3
SD 8.1
MDC95 22.5
```

3行目は、式 $1.96 \times \sqrt{2} \times \text{SEM}$ をそのまま書いたもの（SEMの目安にSDを使う）

帯の外に出たら変わったと言える

- 今回の平均 122.3° の上下 22.5° が、揺れの範囲

次の計測が帯の外に出たら「変わった」と言える



ゴニオメーターと比べる

測り方	SEM	MDC95
ゴニオメーター（肩屈曲、検者内）	2°	5.5°
今日の動画（9回のピークのSD）	8.1°	22.5°

ゴニオメーター：Kolber & Hanney（2012）、健常者30人。MDC95はSEMから上の式で計算

なぜ動画のほうが4倍も大きいのか

ゴニオメーターは「最大まで上げて止めた姿勢」を測る。今日の動画は、本人が最大を狙っていない繰り返しを測っている

揺れは測り方で小さくできる

- 毎回「できるところまで上げて止める」と指示する
- 同じ向き・同じ距離から撮る
- 1回ではなく、数回の平均で比べる

MDCとMCIDは別もの

	問い	決めるもの
MDC	測定の揺れを超えたか	測り方
MCID	患者さんにとって意味のある変化か	患者さんの実感

「良くなった」と言うには、まずMDCを超えている必要がある

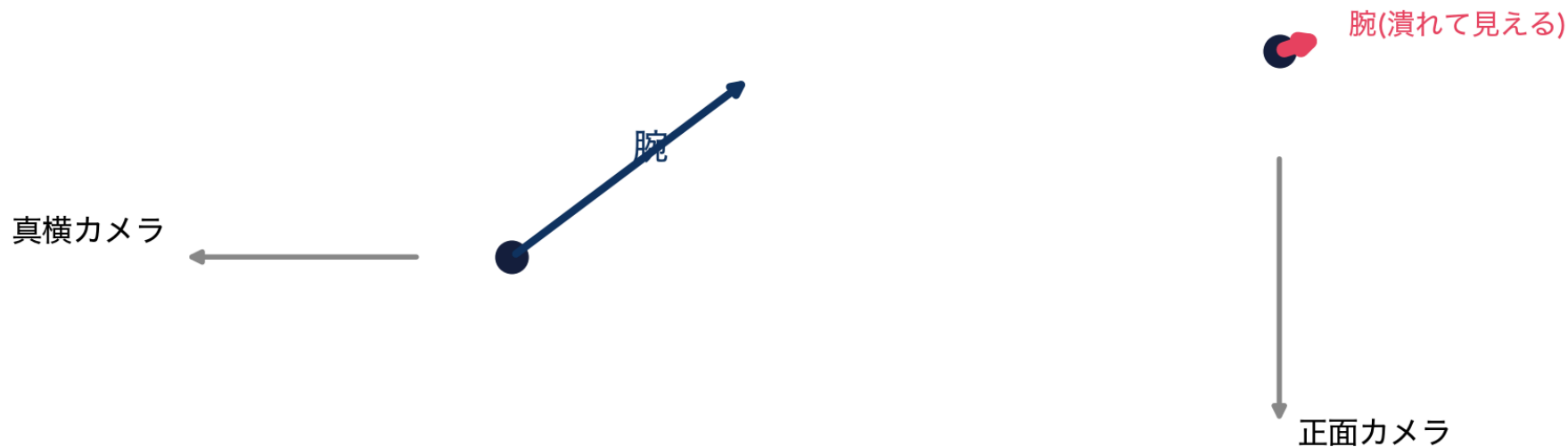
正面から撮ると屈曲が見えない

- 屈曲は前後の動き。正面から撮ると、奥行きの方になって見えにくい

外転動作のカメラアングルによる見え方の違い

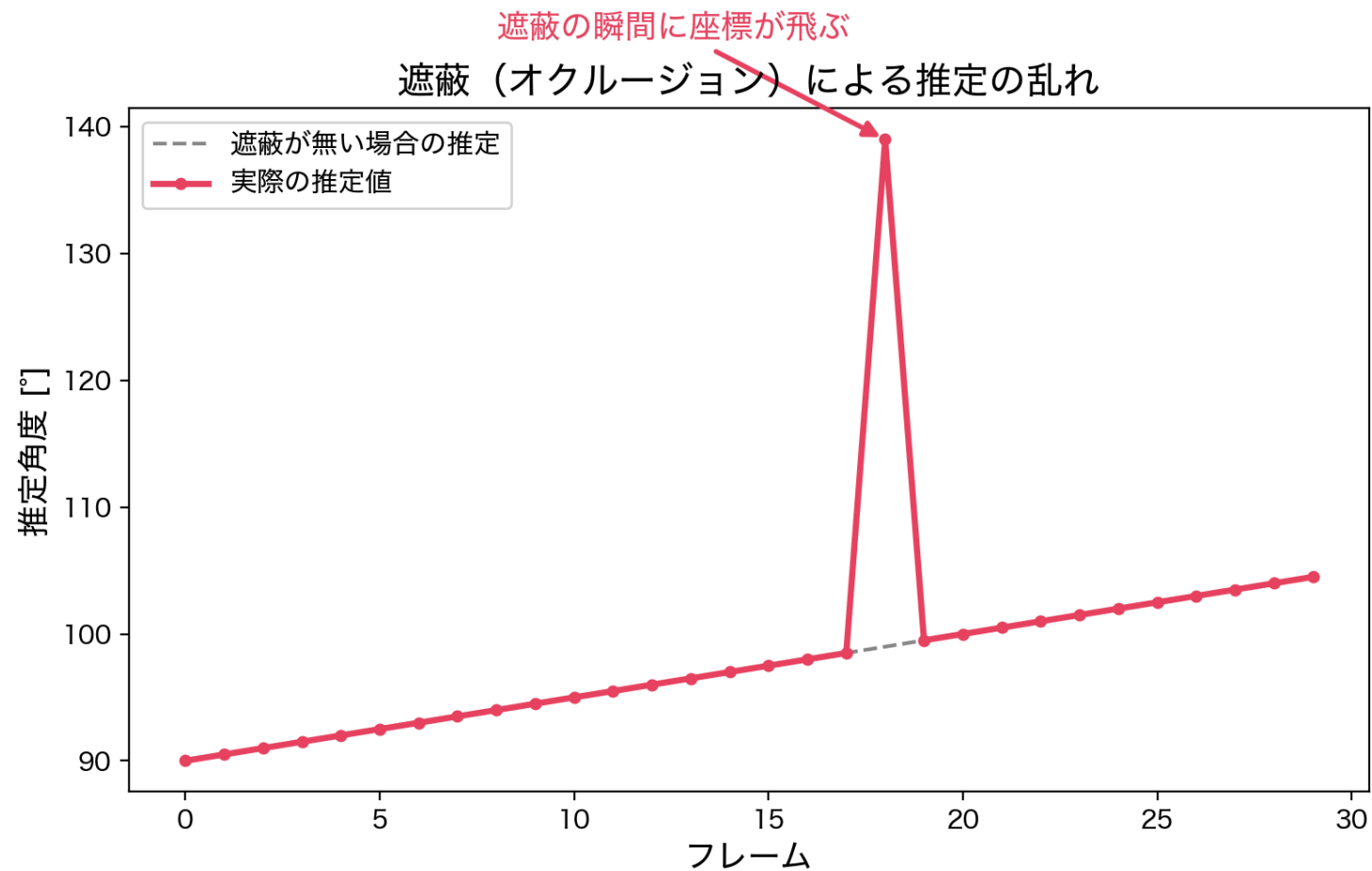
正面カメラ＝腕が潰れる (NG)

真横カメラ＝腕の開きが見える (OK)



隠れた関節は点が飛ぶ

- 腕が体や物の陰に入ると、その間の点の位置が大きくずれることがある（模式図）



ゆったりした服は関節がずれる

- 服の輪郭を体の輪郭と取り違い、関節の位置がずれる（模式図）

服装によるキーポイント検出への影響

タイト＝肘位置正確

ゆったり＝袖を腕と誤認し肘が外へ

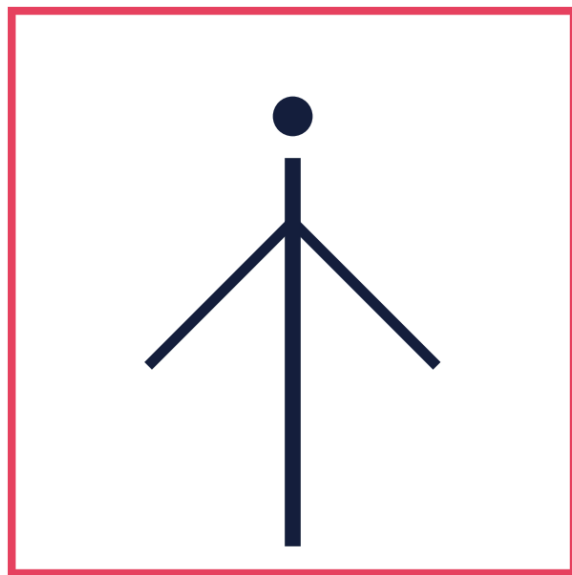


画面からはみ出すと測れない

- 動きの途中で手や頭が画面の外に出ると、その間は推定できない（模式図）

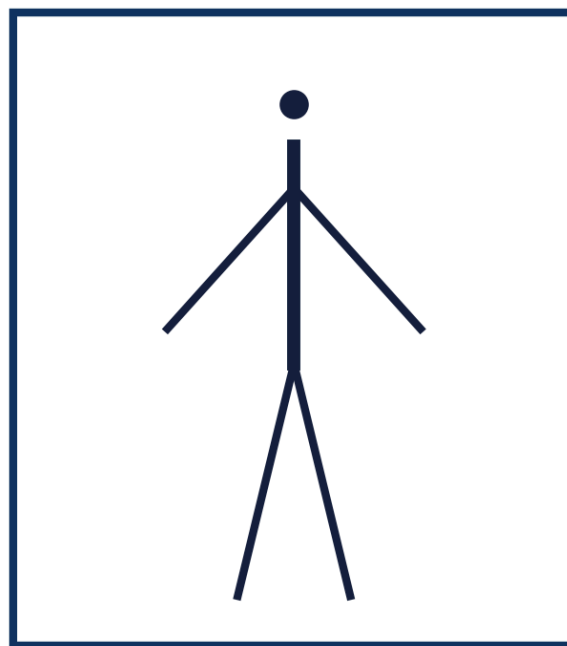
フレーミングによる推定精度への影響

上半身だけ (NG)



上半身だけ（腕・手が枠外へ）

全身+余白 (OK)



全身+余白（動作が枠内に収まる）

撮る前のチェック

項目	そろえ方
向き	屈曲は真横から、外転は正面から
画面	動きの最初から最後まで、全身が入る
服	体の線が分かる服
距離と高さ	毎回同じ場所に、同じ高さで置く

良くなったと言うための条件

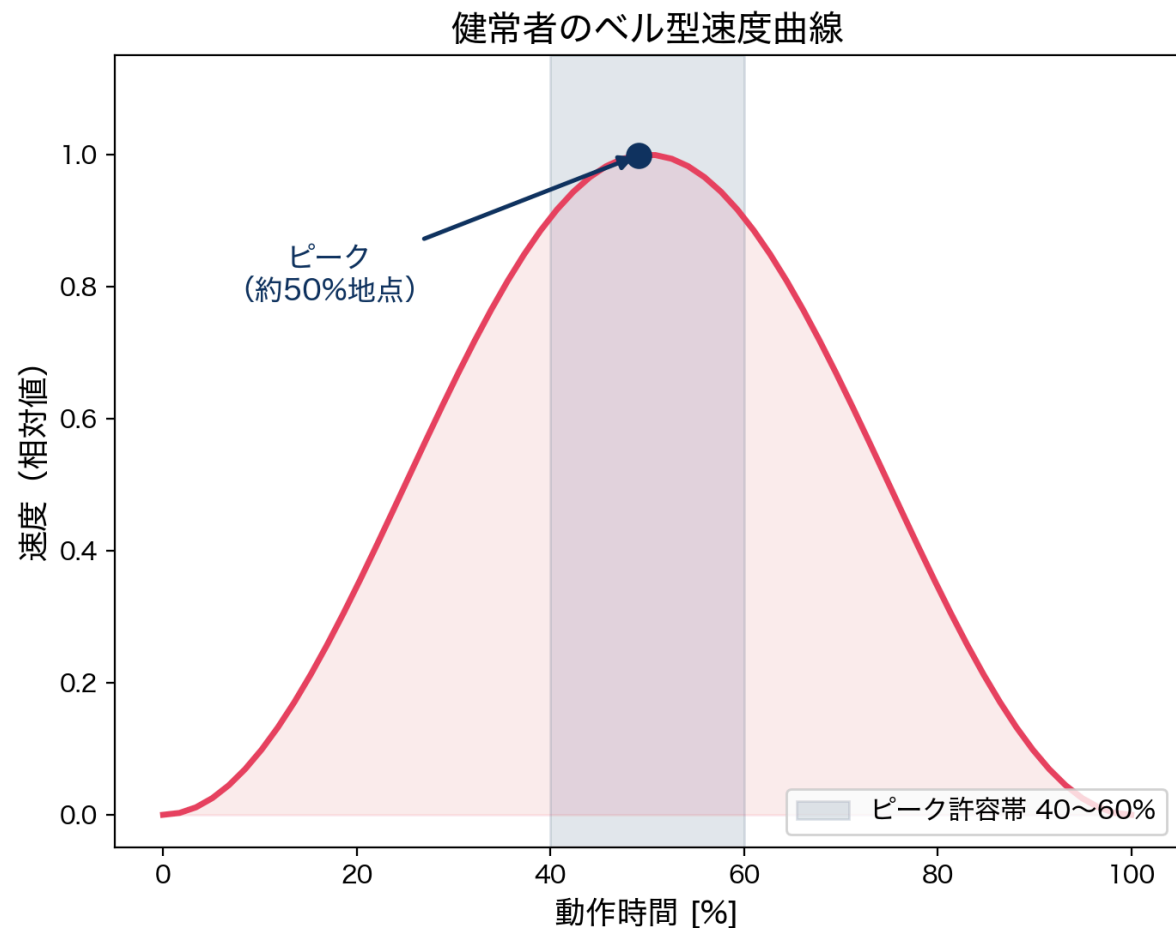
1. 毎回、同じ撮り方・同じ指示で撮る
2. 1回ではなく、数回の平均で比べる
3. 変化が、その撮り方のMDCを超えている

5. どこまで、ではなく、どう動くか

どこまで上がるかは測れた。では、どう動くかは測れるか

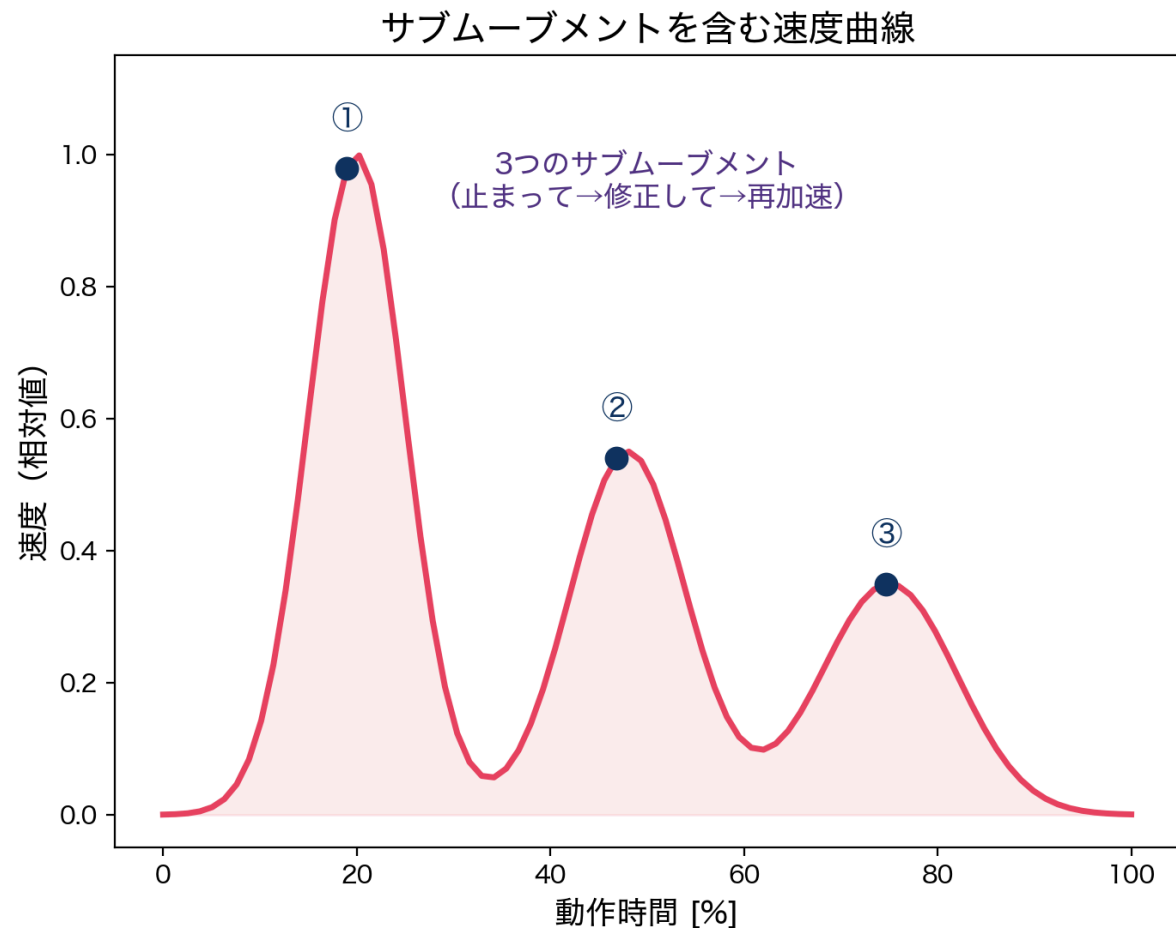
健常な人のリーチはベル型

- 手の速さは、ずっと上がって、ずっと下がる。山は1つ（模式図）



迷うと山が増える

- 止まりかけては動き直すと、山が複数になる。1つ1つの山をサブムーブメントと呼ぶ（模式図）

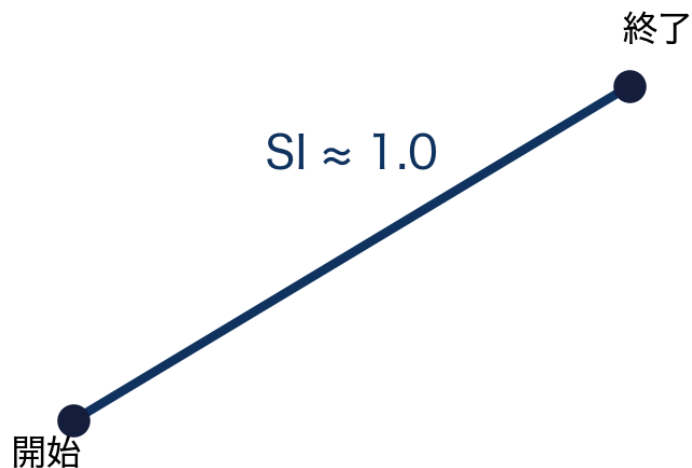


まっすぐさを数にする

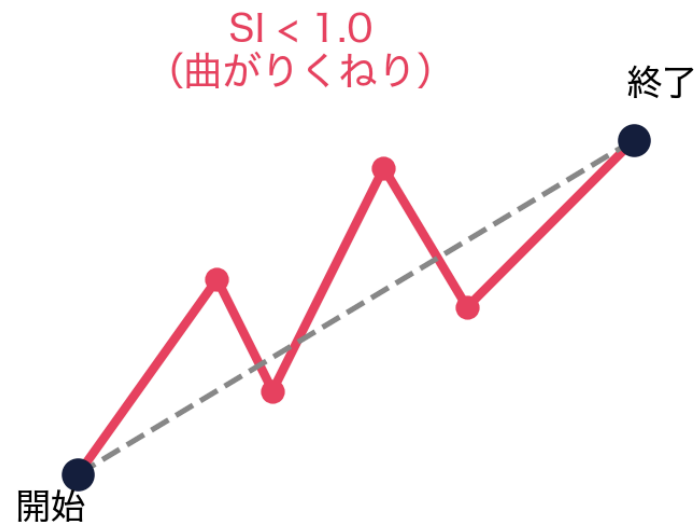
$$SI = \frac{\text{始点から終点までの直線距離}}{\text{実際に通った道のり}}$$

Straightness Index（直線性指数）の直感的比較

直線的な軌跡（健常）



曲がりくねった軌跡（脳卒中後）



速さは距離×コマ数

```
step = np.linalg.norm(np.diff(wrist, axis=0), axis=1)
speed = step * fps
time = np.arange(len(speed)) / fps
plt.plot(time, speed)
plt.xlabel('時間 [秒]')
plt.ylabel('手首の速さ [m/s]')
plt.show()
```

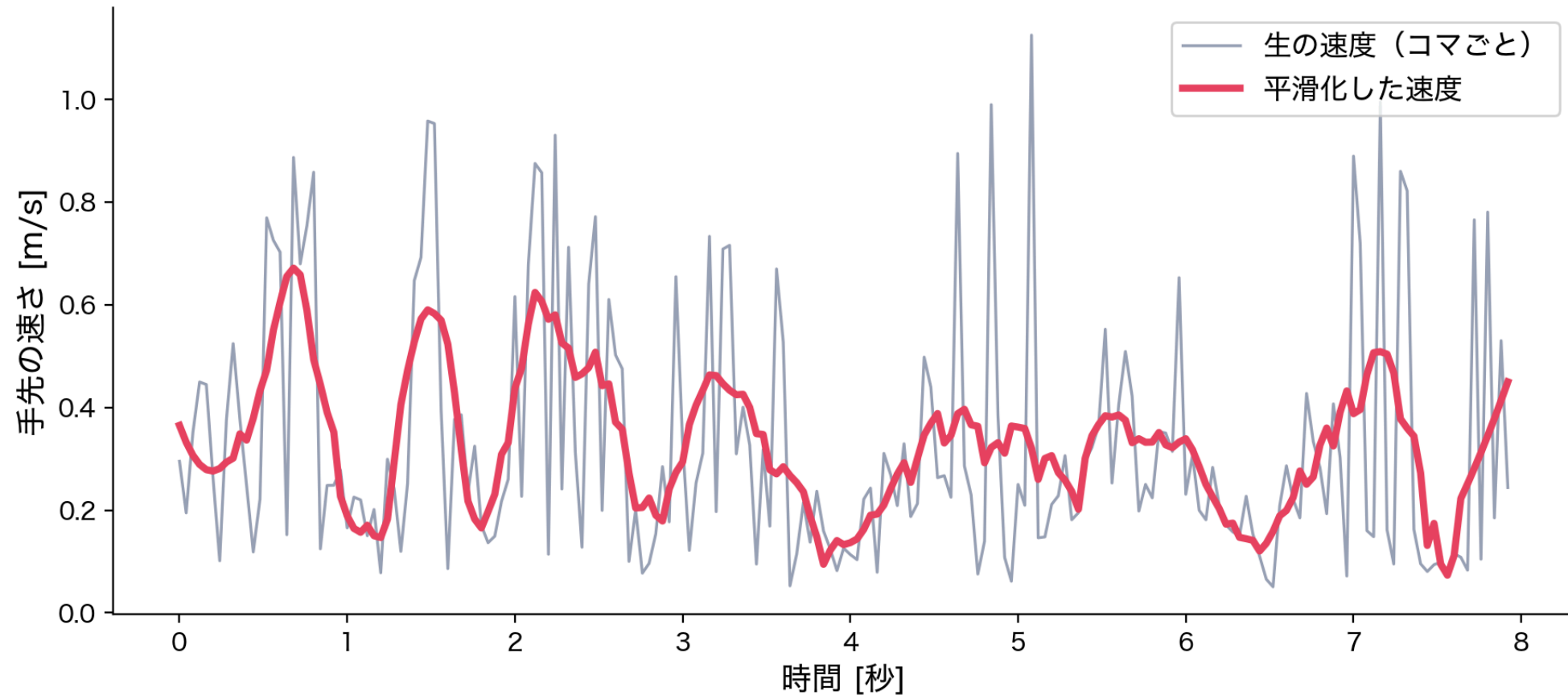
- 1行目：隣どうしのコマで、手首が動いた距離（メートル）
- 2行目：1コマで動いた距離×1秒あたりのコマ数＝1秒あたりに動く距離

実習②のノート。手首の位置`wrist`は、その上のセルで取り出している

生の速さはぎざぎざ

- 灰色の線が、コマごとの速さ。教科書のベル型とはまるで違う

実データ：注ぐ動作8秒の右手首の速さ



ぎざぎざの正体

推定した手首の位置が、コマごとに少しずつ揺れる。その揺れを、速さが大きく拾う

曲線をなめらかにする

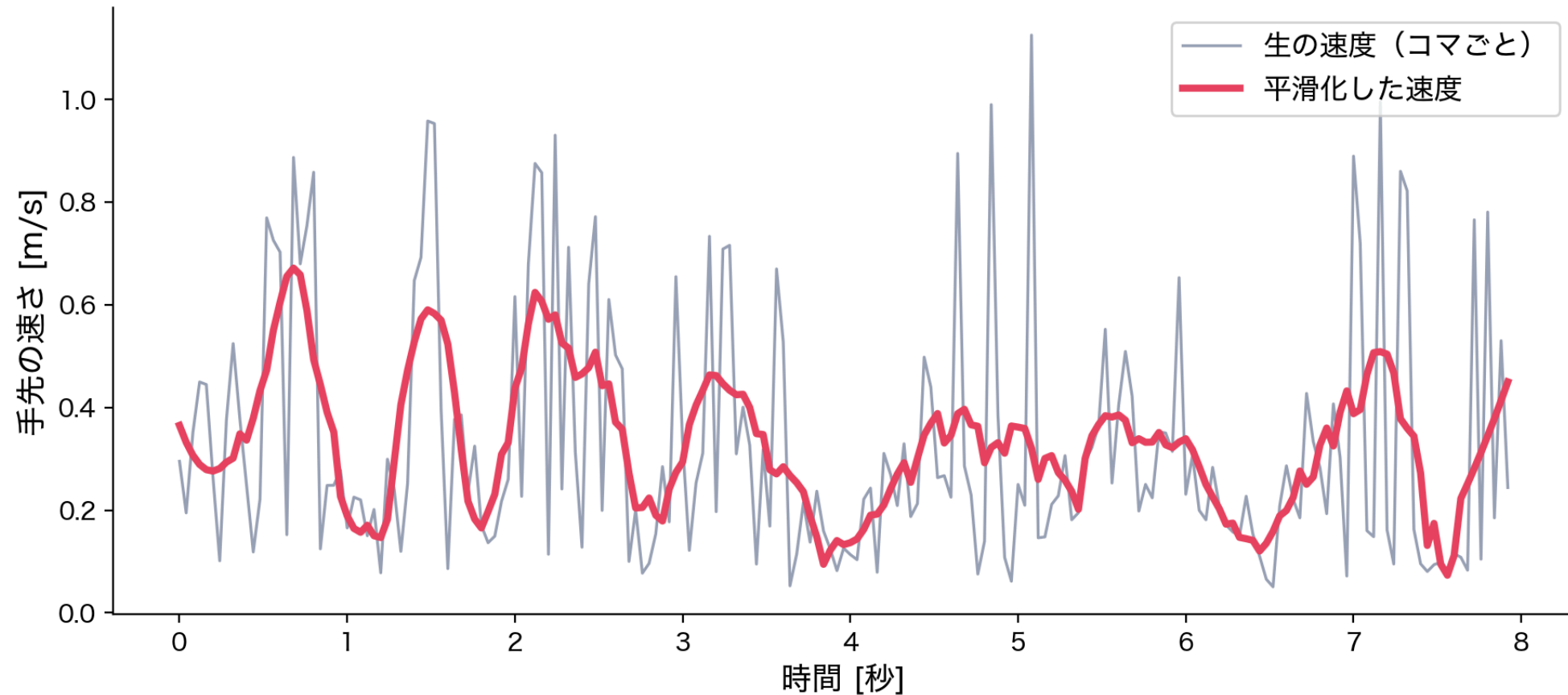
```
speed_smooth = savgol_filter(speed, window_length=15, polyorder=2)
speed_smooth = np.clip(speed_smooth, 0, None)
```

- `savgol_filter` : 前後15コマを使って、曲線をなめらかにする関数
- `np.clip(..., 0, None)` : 0より小さくなった所を0にそろえる（速さは負にならない）

山はいくつある？

- なめらかにした赤い線にも、山がいくつも残る

実データ：注ぐ動作8秒の右手首の速さ



山を数える

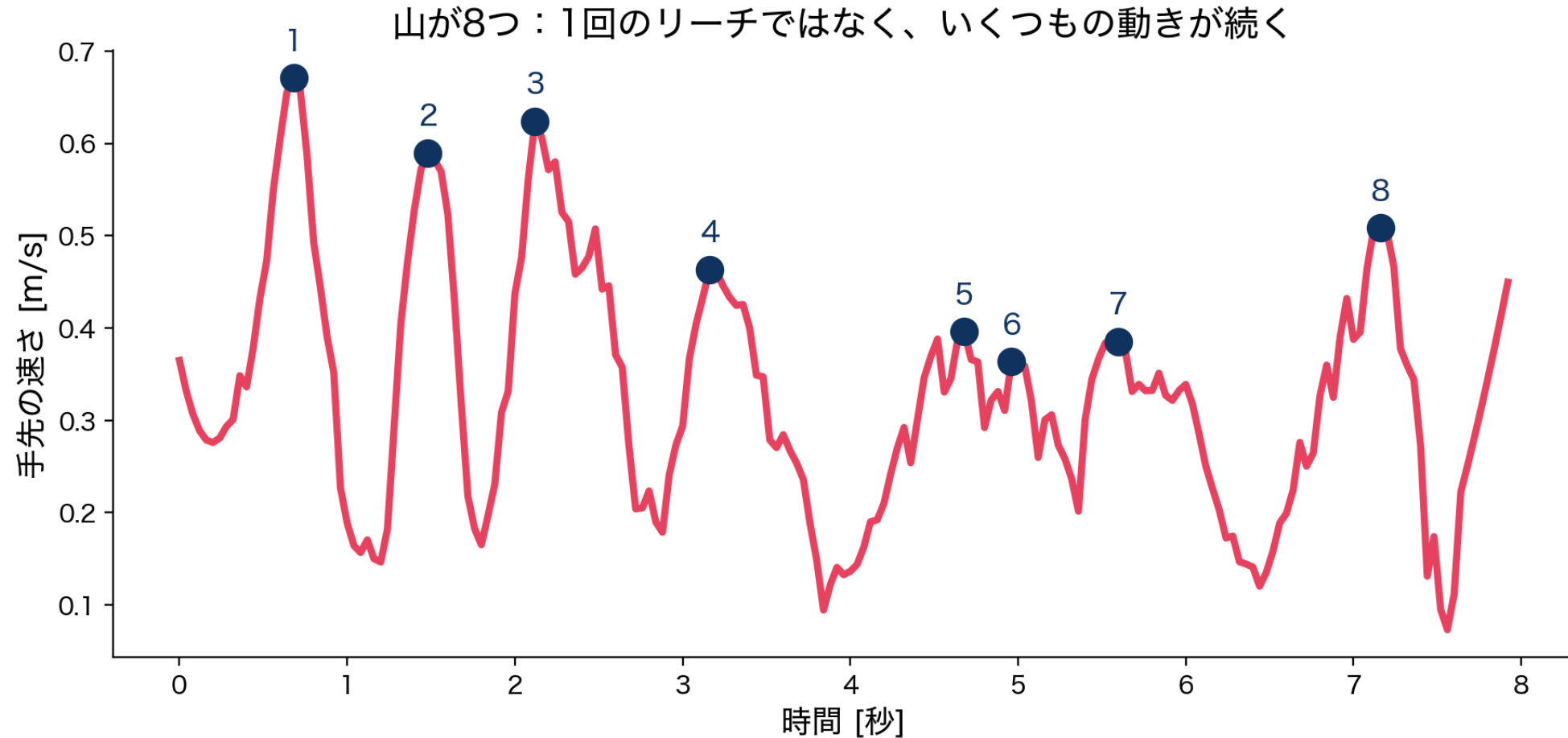
```
top = speed_smooth.max()
peaks, _ = find_peaks(speed_smooth, height=top * 0.1,
                      prominence=top * 0.1)
print('いちばん速いとき (m/s) ', round(top, 3))
print('山の数', len(peaks))
```

```
いちばん速いとき (m/s)  0.671
山の数 8
```

実習①で挙上のピークを探したのと同じ道具。いちばん速い山の10%より低い山は数えない

8つの山は迷いの印か

- 8秒の中に、水を注ぐ・グラスを手にする、が続いている



直線性も出してみる

```
path = step.sum()
straight = np.linalg.norm(wrist[-1] - wrist[0])
print('道のり (m) ', round(path, 3))
print('直線距離 (m) ', round(straight, 3))
print('直線性SI', round(straight / path, 3))
```

```
道のり (m)  2.631
直線距離 (m)  0.196
直線性SI  0.075
```

0.075は1回のリーチの値ではない

8秒の動き全部を「1回の動き」とみなして出した値だから

どこからどこまでを1回の動きとするかで、指標の値が決まる

やってみよう：実習②

1. 実習②のノートを開き、配られたIDを貼って、上から全部押す
2. 速さの曲線と、山の数・直線性SIを確かめる
3. ノートの最後の問いに、メモで答える

右と左を比べる

いちばん速いとき 右 0.671 左 0.641 LSI 104.7
山の数 右 8 左 6 LSI 133.3
直線性SI 右 0.075 左 0.052 LSI 144.2

- 左右差の指数 $LSI = \text{評価したい側} \div \text{反対側} \times 100$ (%)
- 左右の手が同じ課題をしているとは限らないので、この差を麻痺の左右差としては読めない

1回の動きを測るには

- 課題を決める：「目の前のコップに手を伸ばす」を1回ずつ
- 区切りを決める：手が動き出してから止まるまで
- 同じ課題を数回撮り、4節と同じく揺れの幅を出す

まとめ

今日の問い

スマホ動画1本から、この上肢は良くなったと言えるか

言えること

- 動画1本から、全部のコマの肩・肘の角度を出せる
- 今日の撮り方なら、ピークの平均が 22.5° を超えて変われば「変わった」と言える
- 撮り方と指示をそろえれば、この幅は小さくできる

まだ言えないこと

- 本人の揺れと推定の揺れは、今日のデータでは分けられていない
- 動画の角度が、ゴニオメーターの値とどれだけ一致するかは確かめていない
- リーチの質は、1回ずつ区切って撮らないと測れない

持ち帰る1文

同じ撮り方で何度か測り、揺れの幅を超えた変化だけを「良くなった」と言う

配布物

もの	使い方
本編のスライド（PDF）	今日のスライド
発展編のスライド（PDF）	仕組み・信頼性・架空の症例・用語集
実習①・②のノート	Colabで開き、上から全部押す
発展のノート	代償・協調・軌跡を見る（任意）
サンプルデータ2本	ノートの2のセルで読み込む
自分の動画で試す手順	発展編のEと、事前準備の手順書